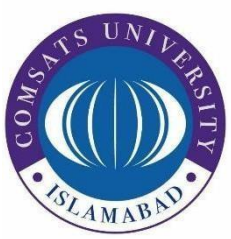

COMSATS University Islamabad, Pakistan,

Sahiwal Campus

NORTH PAK App

A Project Presented To

COMSATS University Islamabad,
Sahiwal Campus

In partial fulfillment

of the requirement for the degree of

Bachelor of Science in Computer Science (2022-2026)

By

Rameen Sheikh CIIT/FA22-BCS-100/SWL

Uzain Farooq CIIT/FA22-BCS-06/SWL

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Muhammad Uzain Farooq

Rameen Sheikh

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (CS) **"NorthPak"** was developed by **Rameen Sheikh (CIIT/FA22-BCS-100) and Muhammad Uzain Farooq (CIIT/FA22 BCS-086)** under the supervision of **"DR Zafar Iqbal Roy"** and co supervisor **"Mam Tahreem Saeed"** and that in (their/his/her) opinion; it is fully adequate, in scope and quality for the degree of Bachelor of Science in Computer Sciences.

Supervisor
Dr. Zafar Iqbal Roy
Assistant Professor
Department of Computer Science
COMSATS University Islamabad, Sahiwal Campus

External Examiner
Dr Ansar Munir Shah
Assistant Professor
Department of Computer Science
University of Southern Punjab Multan

Head of Department
Dr. Zafar Iqbal Roy
Assistant Professor
Department of Computer Science
COMSATS University Islamabad, Sahiwal Campus

Executive Summary

Tourism in the northern areas of Pakistan has grown rapidly in recent years, attracting a large number of local and international visitors. However, travelers still face major challenges such as scattered information, unreliable accommodation details, and the lack of a centralized platform for managing travel activities. Most tourists rely on social media and informal sources, which are often incomplete or inaccurate, making trip planning inefficient and sometimes risky.

To address these issues, the **NorthPak Travel Application** is developed as a smart, centralized digital solution that brings all travel services into one platform. It eliminates the need to search across multiple sources for tours and hotel bookings by providing listings, destination exploration, and seamless booking options in a single application. The system also includes integrated travel tools such as weather updates, budget estimation, distance calculation, location services, and personal travel notes, travel checklist enabling users to plan their trips efficiently. Additional features such as ready to go travel packages, emergency contact information, and a review and feedback system further enhance decision-making and user convenience. The platform also supports local businesses by allowing small and medium sized service providers to connect directly with travelers, increasing their visibility and growth opportunities.

An administrative panel is incorporated to manage bookings, hotel and tour listings, room availability, FAQs, and user reviews. A booking approval mechanism (pending, accepted, or rejected) is implemented before payment to ensure reliability and control. The system is developed using modern mobile and cloud-based technologies, ensuring real-time data synchronization, secure authentication, and smooth performance.

Overall, the **NorthPak Travel Application** provides a user friendly, reliable, and scalable solution for digital tourism management. It simplifies travel planning, improves access to accurate information, and contributes to the promotion of tourism and economic growth in northern Pakistan.

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor **"Dr. Zafar Iqbal Roy"** and our co Supervisor **"Miss Tahreem Saeed".** Without their personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Muhammad Uzain Farooq

Rameen Sheikh

Abbreviations

SRS	Software Require Specification
PC	Personal Computer
FYP	Final Year Project
SDD	Software Design Description

Table of Contents

List of Figures

List of Tables

Chapter 1
Introduction

1. Introduction

Tourism in the northern regions of Pakistan has gained immense popularity due to its natural beauty, scenic landscapes, and cultural diversity. However, planning a trip to these areas can be challenging due to the lack of reliable information, difficulty in booking hotels and tours, and absence of a centralized platform. Travelers often rely on scattered sources such as social media, which may not always provide accurate or complete information. [1]

The NorthPak Travel Application is designed to address these challenges by offering a centralized, user-friendly platform for managing all travel related activities. The application enables users to book hotels and day tours, explore destinations, and access essential travel tools such as weather updates, budget estimation, distance calculation, location services, and travel notes. It also provides ready-to-go travel packages, emergency contact information, and a review system to enhance user decision-making.

Furthermore, the system includes an admin panel that manages bookings, hotel and tour listings, FAQs, user reviews and room availability. The booking approval mechanism ensures that requests are verified before payment, improving reliability and trust. Overall, this application aims to simplify travel planning, promote tourism in northern Pakistan and provide a complete digital solution for travelers.

1.1 Brief

The project, **NorthPak Travel Application**, is a mobile-based solution developed to simplify travel planning for tourists visiting the northern areas of Pakistan. The main outcome of this work is a fully functional application that allows users to book hotels and tours, explore destinations and public reviews, and use integrated travel tools such as weather updates, budget estimation, distance calculation, location services, and travel notes and checklist. The system also includes ready to go travel packages, emergency contact information, and a review system, providing a complete and user-friendly travel management experience.

The application is developed using modern technologies, including **Android Studio**, **Flutter (Dart)** for front end development, and **Firebase** for backend services such as authentication, real-time database, and cloud storage.

1.2 Relevance to course Module

The NorthPak Travel Application is a modern mobile based solution designed to facilitate tourists in exploring northern areas of Pakistan. It provides users with features such as destination browsing, hotel booking, tour packages, and travel assistance in a single platform. The application aims to enhance the overall travel experience by offering real time information, user friendly navigation, and personalized recommendations. By integrating modern technologies like Firebase and mobile app development frameworks, the system ensures efficient data management and smooth performance.

i. Database Systems

Knowledge of Database Systems is utilized through Firebase, which manages real-time data such as user information, bookings, hotel listings, and reviews, ensuring efficient storage and retrieval.

ii. Software Engineering

The project also reflects principles of Software Engineering, including requirement analysis, system design, development, testing, and maintenance, following a Hybrid Software Development Methodology that combines Incremental, Agile, and Iterative approaches for continuous improvement.

iii. Data Structures

In addition, concepts of Data Structures are used to efficiently organize and manage data such as travel packages, hotel records, and user activities, improving the performance of the application.

iv. Parallel and Distributed Computing (PDC)

The project also incorporates Parallel and Distributed Computing concepts through the use of cloud based services like Firebase, which handle multiple user requests simultaneously and ensure real time data synchronization

v. Contribution of External API Integration

Furthermore, the application uses limited external integrations to maintain efficiency. A weather service, **OpenWeather API**, is used to provide real-time weather updates, while navigation and location services are handled through **Google Maps** by redirecting users instead of relying on multiple external APIs. This approach reduces complexity, improves performance, and enhances reliability.

1.3 Project Background

Tourism is one of the fastest-growing industries in the world and plays a vital role in promoting cultural exchange, economic development, and international recognition. Pakistan, particularly its northern regions such as Hunza Valley, Skardu, Gilgit, Naran, and Chitral, is famous for its breathtaking landscapes, rich culture, and historical attractions. Every year, these destinations attract a large number of domestic and international tourists. However, despite their popularity, travelers often face challenges such as difficulty in finding reliable information about destinations, accommodations, travel routes, weather conditions, and available tour services. Most of the existing information is scattered across different platforms like websites, social media, and blogs, which can be inconsistent and unreliable.

With the rapid growth of smartphones and mobile technology, digital tourism platforms have become an effective solution for improving travel planning. Based on this need, the **NorthPak Travel Application** is proposed as a mobile-based tourism management system that simplifies the overall travel experience. The main idea behind the project is to provide a centralized platform where users can access all essential travel services in one place, eliminating the need to search across multiple sources for hotel bookings and tour arrangements.

The application allows users to explore destinations, view verified hotel listings, book tours and accommodations, estimate travel budgets, calculate distances, store travel notes, and share reviews with other travelers. It also includes additional features such as weather updates, ready-to-go travel packages, and emergency contact information to ensure safe and well-organized travel planning.

Furthermore, the system includes an administrative panel that enables efficient management of hotel listings, tours, FAQs, review system, travel media hub and user feedback, ensuring that all information remains accurate and up to date.

1.4 Literature Review

Popular travel platforms such as Booking.com, Airbnb, and TripAdvisor provide services like hotel booking, accommodation listings, and user reviews. These applications allow users to explore destinations and make reservations online. Similarly, Google Maps is widely used for navigation, route planning, and location-based services, while weather applications like Open Weather API provide real-time weather updates.

Although these applications offer useful features, they have certain limitations. Most of them focus on specific services only, such as booking or navigation, and do not provide a complete travel solution in one platform. Users often need to switch between multiple apps to plan a single trip, which increases complexity and reduces convenience. Additionally, many international platforms lack localized content and verified information specific to the northern areas of Pakistan.

Recent trends in tourism emphasize **centralized and mobile-based solutions**, where multiple services are integrated into a single application. Modern systems also focus on user-friendly interfaces, real-time data access, and review-based decision-making. Technologies such as cloud computing and mobile development frameworks are widely used to ensure scalability and performance.

The **NorthPak Travel Application** addresses these gaps by providing an all-in-one platform that combines hotel booking, tour management, travel tools, and verified local information in a single system. [2]

1.5 Analysis from Literature Review

Existing travel applications provide useful services but are generally limited to specific functionalities. Platforms like Booking.com and Airbnb mainly focus on hotel and accommodation booking, while TripAdvisor emphasizes reviews and recommendations. Similarly, Google Maps is widely used for navigation and route planning. Although these applications are effective in their respective areas, they do not provide a complete, integrated solution for travel planning. One of the major limitations of these systems is the lack of a centralized platform. Users are required to switch between multiple applications to manage different aspects of their trip, such as booking hotels, checking routes, viewing weather updates, and planning budgets. This increases complexity and reduces efficiency. Moreover, these applications are mostly global and do not specifically focus on the northern areas of Pakistan.

The Table 1.1 presents a comparative analysis of existing travel applications and the proposed NorthPak Travel Application. It highlights the key limitations of widely used platforms such as Booking.com, Airbnb, TripAdvisor, and Google Maps, including lack of integration, limited functionality, and absence of localized services. This comparison clearly shows that the proposed system offers a more comprehensive, efficient, and user-friendly solution for travel planning in the northern areas of Pakistan. Unlike existing platforms, NorthPak combines destination information, hotel booking, tour packages, travel tools, weather updates, and location-based services within a single application

Table 1.1 Analysis from Literature Review

Application Name	**Weakness**	**Proposed Project Solution**
Booking.com	Focuses mainly on hotel booking; lacks tour planning, travel tools, and personalized trip management; limited localized data for northern Pakistan.	Provides both hotel and tour booking in one platform along with travel tools such as budget estimation, weather updates, and distance calculation, specifically tailored for northern areas of Pakistan.
Airbnb	Limited to accommodation services; lacks complete travel planning features; no integrated tools for trip management.	Offers a complete travel solution including bookings, ready-made packages, travel notes, and integrated planning tools within a single application.
TripAdvisor	Primarily review-based; limited direct booking options; requires users to switch to other platforms for complete planning.	Combines reviews with booking functionality, allowing users to explore, decide, and book within the same platform without switching apps.
Google Maps	Provides only navigation and location services; no booking or travel management features.	Integrates travel planning features and redirects to maps only when needed, reducing dependency while keeping the system simple and efficient.

1.6 Methodology and Software Lifecycle for This Project

NorthPak uses a hybrid SDLC approach combining:

- **Incremental Model (Primary)**
- **Agile Model (Development)**
- **Iterative Model (Internal Refinement)**

This combination ensures structured, flexible, and high-quality development. [3]

1. Incremental Software Development Model (Primary Model)

The **Incremental Model** is used as the primary development approach for NorthPak because the application is naturally divided into clear and independent modules. Each module is developed as a separate increment, allowing early delivery, reduced risk, independent development, easy scalability, and smooth integration with Firebase.

Increment 1
Firebase Authentication is implemented, and the Explore module is developed to display major northern cities with basic attraction details and navigation.
Outcome: A functional application shell with login and city browsing features.

Increment 2
Users can browse hotels, check room availability, book tours, save items to a wishlist, and submit reviews. All bookings are stored in Firestore with a **Pending** status.
Outcome: A complete and interactive booking experience.

Increment 3
An admin panel is developed to manage hotels, rooms, tours, FAQs, and travel packages. Admins can approve or reject bookings using secure role-based access.
Outcome: A controlled and verified backend system.

Increment 4
Additional features such as Budget Estimator, Travel Notes, Distance Calculator (based on the Haversine formula), weather updates and a basic map view are implemented.
Outcome: Improved user convenience and trip planning capabilities.

2. Agile Methodology

Alongside the Incremental Model, NorthPak adopts the **Agile methodology** to manage development and teamwork effectively. Agile supports short development cycles called sprints, daily stand-up meetings, sprint reviews and continuous feedback.

Agile is particularly suitable for NorthPak because:

- Tourism data (hotels, pricing, packages) frequently changes
- UI/UX requires continuous improvement
- Different modules can be developed in parallel
- Firebase-based features require ongoing testing and updates

This approach ensures that the system remains flexible, fast, and user-centered, while adapting easily to new requirements.

3. Iterative Model

The **Iterative Model** is applied within each increment to refine and improve system components. Every module undergoes repeated cycles of:

- Planning
- Design
- Implementation
- Testing
- Feedback
- Refinement

This process continues until the module reaches the desired level of quality and stability.

The iterative approach helps to:

- Continuously improve UI/UX
- Enhance booking workflows and system performance
- Allow easy modifications after testing

1.6.1 Rationale behind Selected Methodology

The **Incremental Model** was chosen because:

- The application consists of multiple independent features (booking, tools, admin panel)
- It allows step-by-step development and easier debugging
- Each module can be tested before integrating into the system

The **Agile Model** was selected because:

- The project requires flexibility due to evolving travel-related features
- It supports continuous feedback and improvement
- It improves collaboration and speeds up development

The **Iterative Model** was used because:

- It helps in refining complex features like booking management and availability handling
- It ensures better user experience through repeated testing
- It improves system reliability and performance

Overall, this combined approach ensures that **NorthPak** is developed as a **scalable, flexible, and user-friendly travel application**

Chapter 2

Problem Definition

2. Problem Definition

This chapter presents a clear description of the problem addressed by the system. It explains the existing issues in travel planning for the northern areas of Pakistan and defines the need for an integrated solution. The chapter also outlines the expected outcomes, including a user-friendly application that enables hotel and tour booking, provides travel tools, and enhances the overall travel experience**.**

2.1 Problem Statement

Tourism in the northern areas of Pakistan has increased significantly in recent years. However, travelers still face many difficulties when planning and managing their trips.

Currently, travel planning is **not centralized**. Users have to use different platforms to book hotels, reserve tours, check weather updates, estimate budgets, and calculate distances. This makes the process **time-consuming, confusing, and inefficient**, especially for new travelers.

One of the main problems is the **lack of a simple and unified booking system**. Users often find it difficult to easily book hotels and tours from one place, and managing bookings becomes inconvenient.

In addition, essential travel tools such as **budget estimation, travel notes and checklist, distance calculation, and emergency contact information** are not available in a single platform. This creates difficulties in proper trip planning.

Another important issue is that many modern applications rely heavily on AI-generated content, which may sometimes provide **inaccurate or misleading information**. There is a need for a system that **limits unnecessary AI usage and focuses on providing authentic and reliable information** to users.

From the management side, there is also a need for a system that allows administrators to **handle bookings, manage hotels and tours, control room availability, and update system data efficiently**.

Therefore, there is a need for a **simple, centralized, and user-friendly travel application** that focuses on easy booking and authentic information,

2.2 Deliverables and Development Requirements

The **NorthPak Travel Application** project involves multiple deliverables produced throughout the development process to ensure proper planning, implementation, and evaluation of the system. These deliverables include both **documentation and a functional software product**, which together represent the complete development lifecycle of the project.

A major deliverable is the **fully functional mobile application**, which implements the core features of the system. The application allows users to explore tourist destinations in the northern areas of Pakistan, book hotels and tours, estimate travel budgets, calculate distances, maintain travel notes, and access essential travel information. It also provides a simple and centralized booking experience along with useful travel tools.

In addition, the system includes an **administrative dashboard** that enables administrators to manage hotels, tours, FAQs, user feedback, and booking requests (approve/reject/pending). This ensures proper control, updated information, and smooth system management.

The development of the system utilizes modern technologies and tools. The **front-end** is developed using Flutter, which allows the creation of a responsive and cross-platform mobile application. The **backend and database services** are managed using Firebase, providing features such as authentication, real-time database, and cloud storage for secure and efficient data handling.

Furthermore, the application integrates external services such as Google Maps API for location-based features and navigation, and weather APIs to provide real-time weather updates for different destinations. Development tools like Android Studio and Visual Studio Code are used for coding, testing, and debugging.

Finally, the project deliverables also include testing results, system demonstrations, and a final presentation, which evaluate the system's performance, usability, and functionality. These deliverables ensure that the project meets its objectives and provides a reliable and efficient solution for travel planning and tourism management.

2.3 Current System

At present, there is no dedicated centralized system specifically designed for managing tourism information in the northern regions of Pakistan. Most travelers rely on multiple independent sources to gather information about destinations, accommodations, and travel services. These sources include social media platforms, travel blogs, tourism websites, and general travel applications. However, this information is often scattered and not tailored to the specific needs of tourists visiting northern Pakistan.

Many travelers use global platforms such **as TripAdvisor, Booking.com, and Airbnb** to search for hotels, explore destinations, and make bookings. While these platforms provide useful features like accommodation booking and user reviews, they primarily focus on international markets and often lack detailed, region-specific information about smaller destinations and local tourism services in northern Pakistan.

In addition, navigation tools such as **Google Maps** are widely used to find routes, distances, and nearby locations. Although these tools are effective for navigation, they do not provide integrated travel planning features such as local tour booking, budget estimation, or complete tourism guidance within a single platform.

Another major limitation of the current system is the **lack of integration** between different services. Travelers often need to switch between multiple applications or websites to book hotels, check weather conditions, plan routes, and manage their trips. This fragmented approach makes the process **time-consuming, inconvenient, and sometimes unreliable**.

Furthermore, many local hotels and tour operators in northern Pakistan lack proper digital platforms to promote their services or manage bookings efficiently. As a result, tourists may face difficulty in discovering local accommodations, cultural experiences, and travel activities available in these areas.

Due to these limitations, the existing system does not fully meet the needs of tourists. This gap highlights the need for a specialized tourism application like **NorthPak**, which aims to provide a **centralized, user-friendly platform**.

Chapter 3

Requirement Analysis

3. Requirement Analysis

Requirement Analysis is the process of identifying and understanding the needs and features required for the application. In this phase, the functionalities, user requirements, and system behavior of the travel app are analyzed.

3.1 Use case Diagram

A Use Case Diagram visually represents the interaction between users and the system. It shows the main features of the travel app such as trip booking, hotel search, travel planning, and admin management through different actors. [4]

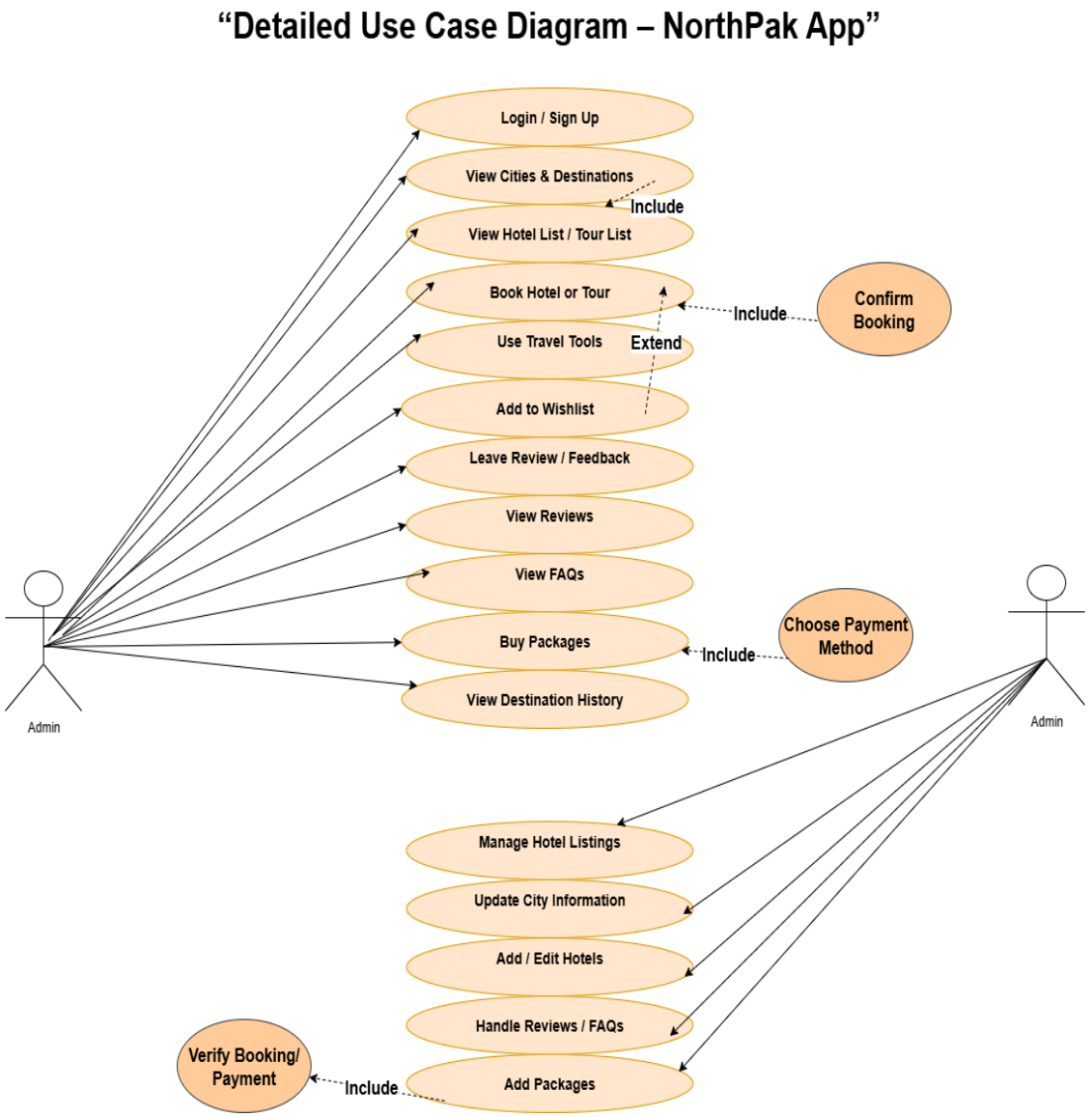

Figure 3.1 Use Case Diagram

3.2 Detailed use case

Detailed Use Cases describe the step-by-step interaction between users and the system. They help in understanding functional requirements and ensure that all system processes are clearly documented.

UC-1: Login / Sign Up

The table 3.1 shows this use case allows the traveler to create a new account or log into an existing account to access the application. The user enters valid credentials such as email and password for authentication. The system verifies the provided information using secure authentication mechanisms. Upon successful login or registration, a user session is created.

Table 3.1 UC-1: Login / Sign Up

Field	**Description**
Use Case ID	UC-1
Use Case Name	Login / Sign Up
Primary Actor	Traveler
Secondary Actor	Admin
Brief Description	Traveler signs up or logs in to access the app.
Trigger	Traveler opens app and selects Login/Sign Up.
Preconditions	1. Internet available 2. User not logged in
Postconditions	1. User session created 2. Traveler moves to Explore page
Normal Flow	1.Open app 2. Choose Login/Sign Up 3. Enter credentials 4. System verifies 5. Dashboard opens
Alternative Flow	Invalid credentials → Show error
Exceptions	Internet disconnected
Business Rules	1. Password ≥ 8 chars 2. Email must be unique
Assumptions	Traveler has valid credentials

UC-2: View Cities & Destinations

The table 3.2 shows this use case allows the traveler to explore different northern area destinations available in the application. The system displays a list of cities along with their associated hotels and tour packages. The traveler can select a specific city to view detailed information and available options. All content is managed and verified by the admin to ensure accuracy. This feature helps users in discovering destinations and planning their trips effectively. If data is unavailable, the system informs the user accordingly.

Table 3.2 UC-2 View Cities

Field	**Description**
Use Case ID	UC-2
Primary Actor	Traveler
Description	Traveler explores northern area destinations and available hotels/tours.
Trigger	Traveler selects "View Cities."
Preconditions	User logged in.
Postconditions	List of cities displayed with hotels/tours.
Normal Flow	1. Traveler opens Explore section. 2. System displays cities. 3. Traveler selects a city. 4. System shows city details, hotels, tours.
Alternative Flow	AF-1: Data unavailable → "Coming Soon."
Business Rules	BR-1: City data verified by Admin.
Assumptions	Admin already uploaded content.

UC-3: Book Hotel or Tour

The table 3.3 shows this use case allows the traveler to book a hotel or cultural/food tour through the application. The user selects a desired listing and provides necessary booking details such as date and number of guests. The system checks availability and processes the request through the payment gateway. Upon successful payment verification, the booking is confirmed and stored in the system. A notification is sent to both the traveler and admin for confirmation.

Table 3.3 UC-3: Book Hotel / Tour

Field	**Description**
Use Case ID	UC-3
Primary Actor	Traveler
Secondary Actors	Payment Gateway, Admin
Description	Traveler books a hotel or cultural/food tour.
Trigger	Traveler clicks “Book.”
Preconditions	PRE-1: Traveler logged in. PRE-2: Listing available.
Postconditions	POST-1: Booking confirmed. POST-2: Payment verified.
Normal Flow	1. Select hotel/tour. 2. System checks availability. 3. Traveler adds details. 4. System confirms booking. 5. Notification sent.
Alternative Flow	AF-1: Booking Rejected → Retry option.
Exceptions	EX-1: Timeout or network loss.
Business Rules	BR-1: Payment required before confirmation.
Assumptions	Booking service operational.

UC-4: Travel Tools

The table 3.4 shows this use case allows the user to access various travel tools provided within the application to assist in trip planning. The system provides real-time weather updates and location-based services using external APIs. The user inputs relevant data, and the system processes. The results are displayed instantly or saved for future use.

Table 3.4 UC-4: Travel Tools

Field	Description
Use Case ID	UC-4
Primary Actor	User
Secondary Actor	Weather API, Maps API, Calculation Module
Description	The user utilizes travel tools including budget estimation, distance calculation, travel notes, location services, trip checklist and weather updates for trip planning.
Trigger	User selects “Travel Tools” from the navigation menu.
Preconditions	User is logged in. Internet connection is available.
Postconditions	Budget is estimated, distance is calculated, notes and travel checklist is saved, and weather and location data is displayed.
Normal Flow	1. User opens Travel Tools. 2. User selects a tool (Budget, Distance, Notes, Checklist, Weather, Location) 3. User enters required inputs (e.g., days, destination). 4. System processes data using APIs or internal logic. 5. System displays results or saves notes successfully.
Alternative Flow	AF-1: API failure → System shows error message. AF-2: Invalid input → System prompts user to correct input.
Business Rules	BR-1: Calculations must use updated travel rates. BR-2: Weather data must be fetched from external API.
Assumptions	External APIs are available and responsive.

UC-5: Add to Wishlist

The table 3.5 shows this use case allows the traveler to save selected hotels or tour experiences for future reference. The user can add items to a personal Wishlist by clicking the "Add to Wishlist" option. The system stores the selected items under the user's account for later viewing. Travelers can easily access and manage their saved items at any time. This feature helps users plan trips by shortlisting preferred options.

Table 3.5 UC-5: Add to Wishlist

Field	**Description**
Use Case ID	UC-5
Primary Actor	Traveler
Description	Traveler saves hotels or tours for later viewing.
Trigger	Click "Add to Wishlist."
Preconditions	Traveler logged in.
Postconditions	Wishlist updated.
Normal Flow	1. Click "Add to Wishlist." 2. System saves item. 3. Traveler can view it later.
Alternative Flow	AF-1: Already saved → No action performed.
Business Rules	BR-1: One Wishlist per tour.
Assumptions	Traveler account active.

UC-6: Leave Review

The table 3.6 shows this use case allows the traveler to provide feedback by writing a review after completing a trip or hotel stay. The user can rate the experience and add comments based on their satisfaction. The system ensures that only users with completed bookings can submit reviews. Once submitted, the review is stored in the system and may be monitored by the admin. This feature helps improve service quality and assists other users in decision-making. Duplicate reviews for the same booking are not allowed.

Table 3.6 UC-6: Leave review

Field	**Description**
Use Case ID	UC-6
Primary Actor	Traveler
Secondary Actor	Admin
Description	Traveler writes a review after a completed trip or stay.
Trigger	Traveler selects “Write Review.”
Preconditions	Traveler must have completed booking.
Postconditions	Review stored in system.
Normal Flow	1. Open App. 2. Enter rating & comment. 3. Submit review. 4. Review gets verified by admin before publish.
Alternative Flow	AF-1: Duplicate review not allowed.
Business Rules	BR-1: One review per booking.
Assumptions	Reviews monitored by Admin.

UC-7: Buy Packages

The table 3.7 shows this use case allows the traveler to purchase a travel or hotel package through the application. The user selects a desired package and proceeds to choose a preferred payment method. The system processes the payment using an integrated payment gateway. Upon successful payment verification, the booking is confirmed and recorded in the system. A confirmation message is displayed to the traveler. In case of payment failure, the system allows the user to retry the transaction.

Table 3.7 UC-7: Buy Packages

Field	**Description**
Use Case ID	UC-7
Primary Actor	Traveler
Secondary Actor	Payment Gateway
Description	Traveler buys a travel or hotel package.
Trigger	Traveler selects “Buy Package.”
Preconditions	Traveler logged in and package available.
Postconditions	Payment verified; booking confirmed.
Normal Flow	1. Select package. 2. Choose payment method. 3. Admin processes payment. 4. Confirmation shown.
Alternative Flow	AF-1: Payment declined → Retry.
Business Rules	BR-1: Payment verification mandatory.
Assumptions	Payment service active.

UC-8: Manage Hotel Listings

The table 3.8 shows this use case allows the admin to manage hotel listings within the application. The admin can add new hotels, update existing details, or remove outdated listings. The system ensures that only authorized admins can perform these actions. All changes are validated before being saved to the database. Once updates are completed, the system confirms the successful modification. In case of invalid input, the system displays an error message for correction.

Table 3.8 UC-8: Manage Hotel Listings

Field	**Description**
Use Case ID	UC-8
Primary Actor	Admin
Description	Admin adds, edits, or removes hotel listings.
Trigger	Admin opens Manage Hotels module.
Preconditions	Admin logged in.
Postconditions	Hotel data updated.
Normal Flow	1. Admin selects Manage Hotels. 2. Adds/edits listings. 3. Saves changes. 4. System confirms update.
Alternative Flow	AF-1: Invalid data → Error prompt.
Business Rules	BR-1: Only verified admins can modify.
Assumptions	Database available.

UC-9: Review FAQs and User Reviews

The table 3.9 shows this use case allows the admin to review and manage FAQs and user reviews available in the system. The admin checks FAQs to ensure accurate information is provided to users and reviews to ensure they follow community guidelines. The system allows the admin to approve, edit, or remove content when necessary. After successful moderation, valid FAQs and reviews are stored in the database and displayed to users. If any content is invalid or inappropriate, the system rejects or removes it.

Table 3.9 UC-9: Review FAQs and User Reviews

Field	**Description**
Use Case ID	UC-9
Primary Actor	Admin
Secondary Actor	Database System
Description	Admin reviews and manages FAQs and user reviews.
Trigger	Admin selects “FAQs or User Reviews.”
Preconditions	Admin logged in.
Postconditions	FAQs and reviews updated successfully.
Normal Flow	1. Admin views FAQs and reviews. 2. Admin checks and manages content. 3. System stores updates.
Alternative Flow	AF-1: Invalid or inappropriate content found → Content removed or edited.
Business Rules	BR-1: FAQs and reviews must follow system guidelines.
Assumptions	Database connection functional.

UC-10: Add Packages

The table 3.10 shows this use case allows the admin to add new travel packages to the system after verifying vendor details. The admin enters package information such as pricing, itinerary, and services offered. Before storing the package, the system ensures that vendor payment and details are properly verified. Only validated packages are added to the database for user access. The system confirms successful addition once verification is complete. If verification fails, the process is stopped and the package is not added.

Table 3.10 UC-10: Add Packages

Field	Description
Use Case ID	UC-10
Primary Actor	Admin
Secondary Actor	Payment Verification Service
Description	Admin adds new packages and verifies vendor payment.
Trigger	Admin selects “Add Package.”
Preconditions	Admin logged in. Vendor registered.
Postconditions	Package added after verification.
Normal Flow	1. Admin enters package info. 2.Admin verifies. 3. System stores package.
Alternative Flow	AF-1: Required package details missing → Package not added.
Business Rules	BR-1: All package information must be provided before submission.
Assumptions	Admin is authenticated and has permission to add packages.

3.3 Functional Requirements

Functional Requirements define the main functions and features of the travel app. They describe the operations the system must perform for users and admins.

FR-1: User Registration and Authentication

Table 3.11 FR-1 User Registration

Field	Description
Identifier	FR-1
Title	User Registration and Authentication
Requirement	Users can sign up and log in using their email and password, while admins have separate login access.
Source	Application Security Requirements
Rationale	Enables secure access and separates user and admin roles.
Business Rule	Passwords must contain at least 8 characters.
Dependencies	FR-3 (Booking System), FR-4 (Wishlist), FR-6 (Admin Management)
Priority	High

FR-2: Explore and Destination Browsing

Table 3.12 FR-2 Explore Destinations

Field	Description
Identifier	FR-2
Title	Explore and Destination Browsing
Requirement	The **user** shall be able to **browse northern cities** and explore it.
Source	Market Analysis
Rationale	Provides users an intuitive way to discover destinations.
Business Rule	Data must be fetched only from verified admin-uploaded listings.
Dependencies	FR-3 (Booking System), FR-5 (Travel Tools)
Priority	High

FR-3: Hotel and Tour Booking

Table 3.13 FR-3: Hotel and Tour Booking

Field	Description
Identifier	FR-3
Title	Hotel and Tour Booking
Requirement	The user shall be able to select a hotel or tour**,** choose stay duration or tour days, and confirm booking. The system shall store booking details securely in the database.
Source	Client requirement and competitor analysis
Rationale	Core functionality allowing travelers to reserve accommodations and tours.
Business Rule	Booking confirmation requires successful payment verification.
Dependencies	FR-1 (Authentication), FR-6 (Admin Management)
Priority	High

FR-4: Wishlist and Review System

Table 3.14 FR-4: Wishlist and Review System

Field	Description
Identifier	FR-4
Title	Wishlist and Review System
Requirement	The user shall be able to add hotels or tours to a Wishlist and post reviews and ratings after completing trips.
Source	User feedback and UX design meeting
Rationale	Enhances user engagement and helps others make informed choices.
Business Rule	One review per completed booking; user must be logged in.
Dependencies	FR-1 (Authentication), FR-3 (Booking)
Priority	Medium

FR-5: Travel Tools

Table 3.15 FR-5: Travel Tools

Field	Description
Identifier	FR-5
Title	Travel Tools
Requirement	The **user** shall be able to **estimate trip costs, calculate distances between destinations**, **create personal travel notes, location services, travel checklist and weather update** for planning.
Source	Enhancement request during system design
Rationale	Provides utility tools to assist travelers in trip preparation.
Business Rule	Calculations depend on updated travel rates and route APIs.
Dependencies	FR-2 (Destination Browsing)
Priority	Medium

FR-6: Admin Management

Table 3.16 FR-6: Admin Management

Field	Description
Identifier	FR-6
Title	Admin Management
Requirement	The **admin** shall be able to **add, edit, or delete hotels, tours, packages, room availability and FAQs**, and **review user comments**.
Source	Backend requirements
Rationale	Ensures up-to-date content and maintains system quality.
Business Rule	Only authenticated admins can modify data.
Dependencies	FR-1 (Authentication), FR-3 (Booking System)
Priority	High

3.4 Non-Functional Requirements

Non-Functional Requirements describe the standards and constraints under which the system operates. They ensure that the application remains secure, reliable, efficient, and user-friendly.

1. Performance

The system is designed to deliver high performance by ensuring fast loading of hotel listings, city information, and booking confirmations within seconds. It supports multiple concurrent users efficiently without any noticeable lag or system slowdown.

2. Security

User credentials are protected using secure hashing algorithms. Authentication and session management are handled through Firebase Authentication, ensuring protection against unauthorized access. All data transmission is secured using HTTPS encryption.

3. Scalability

The application follows a scalable architecture that allows easy expansion. New cities, hotels, and advanced features such as AI-based modules can be integrated through both vertical and horizontal scaling of backend services and databases.

4. Usability

The interface is designed to be simple, intuitive, and user-friendly. Clearly labeled menus, readable icons, and smooth transitions ensure that users can navigate the application easily without confusion.

5. Compatibility

The system maintains full compatibility across major devices and platforms. Responsive design ensures that the application adapts seamlessly to different screen sizes, providing a consistent user experience.

6. Reliability

Reliability is ensured through automated backup and recovery mechanisms. The system is designed to maintain uptime above 99%, minimizing downtime and preventing data loss.

Chapter 4

Design and Architecture

4. Design and Architecture

Design and Architecture define the organization of the system and the relationships between its components. This section explains how the application is structured to deliver its functionality efficiently and reliably.

4.1 System Architecture

The NorthPak system follows a four-layered architecture that divides the application into separate layers based on specific responsibilities. The Figure 4.1 shows how this structure improves organization by clearly separating the user interface, business logic, data handling, and backend services. Each layer works independently while communicating with others in a controlled manner. The use of Firebase in the backend ensures secure and scalable cloud support.

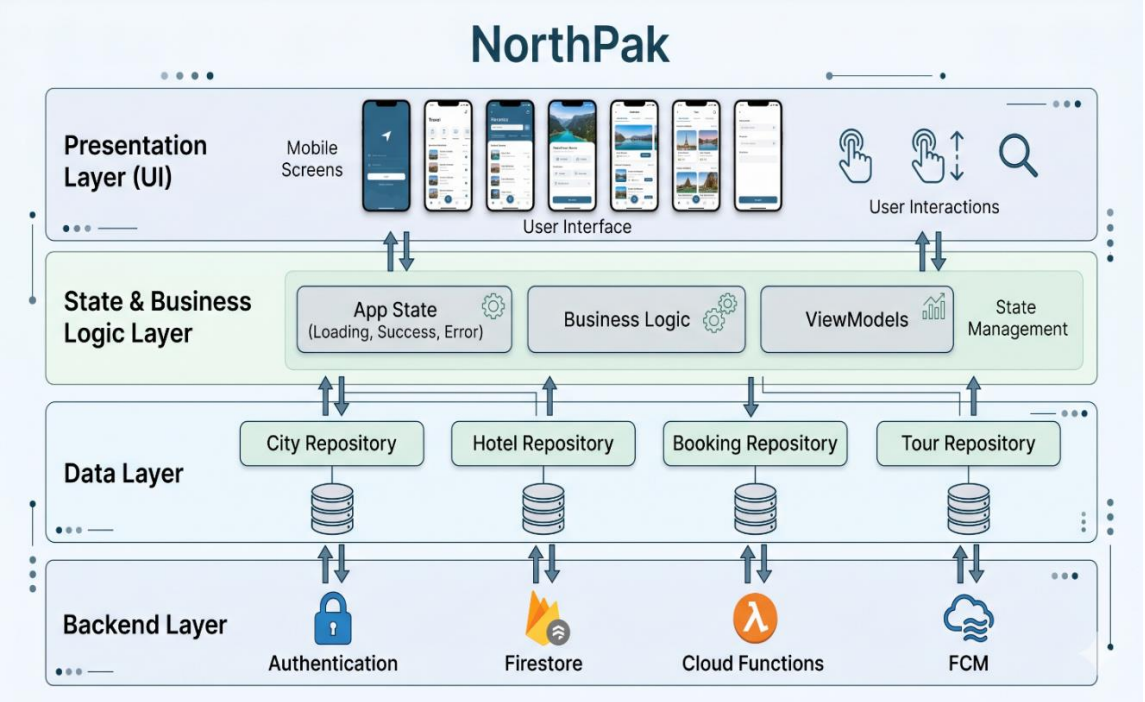

Figure 4.1 System Architecture

1. Presentation Layer

The Presentation Layer is the top most layer of the system and is responsible for all user interactions. It includes screens, user interface components, and navigation flows within the mobile application. This layer captures user inputs such as login details, booking selections, and search queries, and displays relevant outputs like hotel listings, tour packages, and booking confirmations. It does not contain complex logic; instead, it communicates with the underlying layer to process data and update the UI accordingly.

2. State & Business Logic Layer

This layer acts as the brain of the application. It contains ViewModels and controllers that handle application logic, input validation, and state management.

For example, when a user attempts to book a hotel, this layer:

- Validates input data
- Processes business rules (e.g., availability, pricing)
- Manages UI state changes (loading, success, error)

By separating logic from the UI, this layer ensures that the application remains clean, testable, and easier to debug.

3. Data Layer

The Data Layer is responsible for managing all data operations. It acts as an intermediary between the business logic and backend services. This layer uses repositories to organize data access for different entities such as:

- City
- Hotel
- Booking
- Tour

These repositories fetch and store data using Firebase Firestore and may also utilize local storage for caching purposes. This design ensures consistent and structured data handling across the application.

4. Backend Layer

The Backend Layer provides core services required for the application to function. It is implemented using Firebase and includes:

- **Authentication** – Manages user login and registration securely
- **Firestore** – Stores and retrieves real-time application data
- **Cloud Functions** – Executes backend logic such as booking validation or notifications
- **Firebase Cloud Messaging (FCM)** – Sends push notifications to users

This layer ensures secure data storage, real-time updates, and automation of business processes.

4.2 Data Representation

The information domain of NorthPak covering Cities, Hotels, Rooms, Tours, Users, Bookings, Reviews, Packages, and Wishlist items is directly mapped into Firebase Firestore as structured collections and documents. This Figure 4.2 illustrates the simplified data flow architecture of a mobile application, showing how data moves sequentially through different layers of the system. The process begins with the App (UI layer), where the user performs actions such as searching or booking, and these actions generate requests. These requests are passed to the ViewModel, which acts as an intermediary by processing the data and preparing it for further handling. The ViewModel then sends the processed data to the Repository, which is responsible for managing all data operations and deciding how and where the data should be fetched or stored.

Next, the Repository communicates with Cloud Functions, where requests are validated and business logic is applied, ensuring secure and correct processing. Finally, the data is stored and synchronized in Firestore, which acts as the central database.

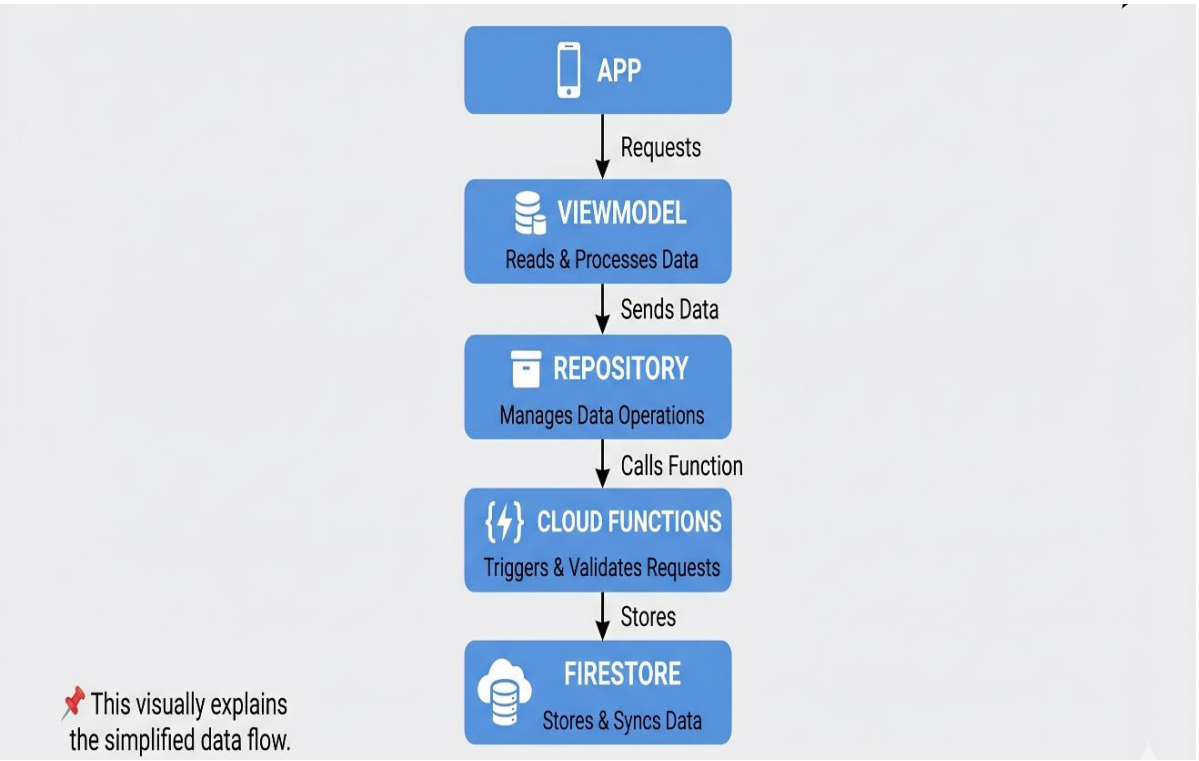

Figure 4.2 Data representation

Data Processing and Organization

Firestore Collections & Sample Schemas

The table 4.1 summarizes the Firestore collections used in the NorthPak system, covering core modules such as cities, hotels, rooms, tours, users, bookings, and reviews. Each collection contains key fields that store essential information and sub-collections. [5]

Table 4.1 Firestore Schema

Collection	Key Fields	Description
cities	id, name, description, lat, lng, topAttractions, createdAt	Stores basic info of northern cities.
hotels	id, name, cityId, address, rating, images, contactPhone, shortDescription, createdAt	Partner hotel details.
hotels/{hotelId}/rooms	id, type, pricePerNight, availableCount, maxGuests, amenities, images	Room types and availability per hotel.
tours	id, title, cityId, type, durationHours, price, description, images, createdAt	Day/cultural/food tours.
users	uid, name, email, phone, profileImage, createdAt	Registered user profiles.
users/{uid}/wishlist	refId, refType, savedAt	User's saved hotels/tours.
users/{uid}/bookings	bookingId	Reference to user booking.
bookings	id, userId, itemId, itemType, snapshots, startDate, endDate, totalPrice, status, createdAt	Stores hotel/tour booking details.
reviews	id, userId, itemId, itemType, rating, comment, createdAt	User reviews for hotels/tours.
packages	id, title, cities, inclusions, price, createdAt	App-exclusive tour/travel packages.

Data Dictionary

The table 4.2 groups commonly used Firestore fields alphabetically and summarizes their purpose in the NorthPak system.

Table 4.2 Data Dictionary

Letter	Fields	Purpose
A	adminActionAt,,adminActionBy,,answer, available	Admin actions, FAQ answers, room availability
B	bookingRules	Package booking rules
C	cancellationPolicy, checkIn, checkOut, city	Booking dates, city info, cancel policy
D	date, day, description	Tours/itinerary dates and descriptions.
E	experience	Experience name.
H	highlight, heroImage, hotel	Tour highlights, banner images selected hotel
I	image, itinerary	Tour and experience images and full itinerary list
N	name, nightStay	User name and night-stay location
O	operator, overview	Operator details and package overview show
P	packageInfo, paymentOptions, phone, price	Package, price, payment, phone number.
R	roomType	Room category (e.g., Deluxe, 2 bed) save.
S	status, subtitle, tagline	Booking status, subtitles.
T	timing, timestamp, title	Timing, record timestamps
U	user, userName	User’s email and name save.

4.3 Process Flow

A process flow diagram provides a detailed understanding of how different components of a system interact with each other. It shows the movement of data, the order of operations, and the decision points where different outcomes may occur. This makes it easier to analyze system behavior, identify inefficiencies, and understand user interactions.

The Figure 4.3 illustrates the workflow and functionality of the travel application for both users and admins. It shows how users can log in, explore hotels and tours, manage Wishlist, and create bookings through different app modules. The system also provides additional features such as travel tools, FAQs, emergency contacts, and profile management. Booking requests are sent to the admin dashboard where the admin can approve or reject them. The diagram demonstrates the interaction between user activities, app modules, and admin management.

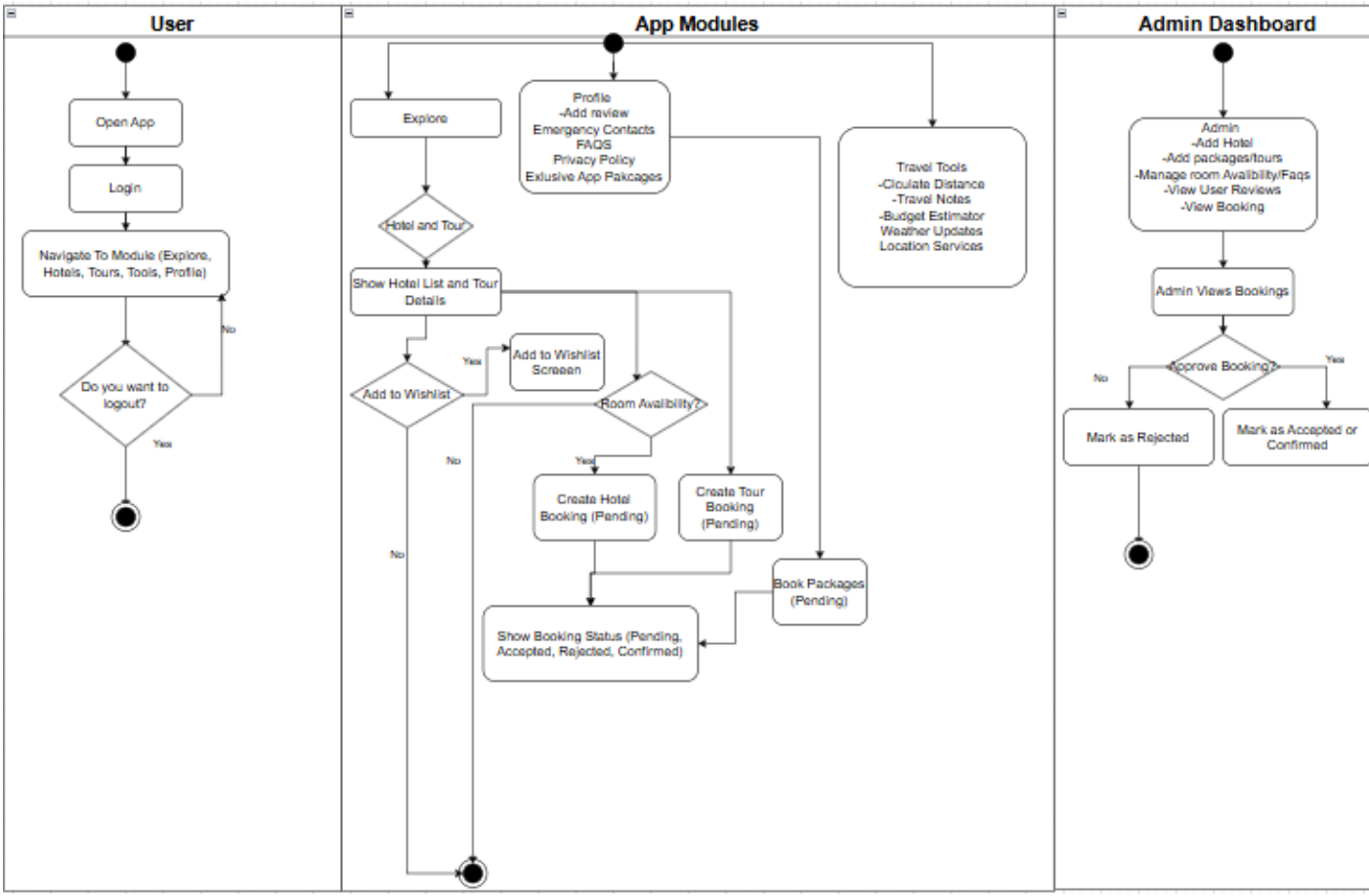

Figure 4.3 App process flow

User Module

The User Module represents the interaction of end-users with the mobile application. It defines how users access the system, navigate between features, and perform actions such as browsing, booking, and logging out. This module focuses on providing a smooth and user-friendly experience.

Description

- The process starts when the user opens the app and logs in.
- After login, the user can navigate to different modules such as Explore, Hotels, Tours, Tools, or Profile.
- The user continues interacting with the app until they decide to log out.
-

App Modules

The App Modules section is the core functional part of the system. It includes all the features that allow users to explore destinations, make bookings, manage preferences, and use travel-related tools. This module acts as the bridge between the user interface and backend services.

Explore & Booking Flow

- Users can explore hotels and tours and travel content.
- The system displays hotel lists and tour details.
- Users can:
    - Add items to **Wishlist**
    - Check **room availability**
    - Proceed with **bookings**
    - View **Travel Content**

Booking Process

- If available:
    - Create **Hotel Booking (Pending)**
    - Create **Tour Booking (Pending)**
    - Or **Book Packages**
- Booking status is shown as:
    - Pending
    - Rejected
    - Confirmed

Wishlist Feature

- Users can save hotels/tours to a **Wishlist screen** for later access.

Profile Features

- Add **reviews**
- Manage **emergency contacts**
- Access **FAQs and privacy policy**
- View **exclusive packages**

Travel Tools

- Calculate **distance**
- Access **travel notes**
- Use **budget estimator**
- View **weather updates**
- Enable **location services**
- Make your **travel checklist**

Admin Module

The Admin Module is responsible for managing and controlling the system from the backend. It allows administrators to maintain data, monitor bookings, and ensure smooth operation of the platform.

Description

- Admin can:
    - **Add hotels and packages/tours**
    - Manage **room availability and FAQs**
    - View **user reviews**
    - Monitor **bookings**

Booking Management

- Admin views all booking requests.
- Decision point: **Approve Booking?**
    - If **Yes** → Mark as Accepted/Confirmed
    - If **No** → Mark as Rejected

4.4 Design Models

The design models of the NorthPak system illustrate how various components interact to deliver complete application functionality. These models provide both a visual and structural representation of system behavior, data flow, and communication between different modules. They act as a comprehensive blueprint for developers during implementation, ensuring consistency, modularity, and ease of maintenance throughout the development lifecycle.

NorthPak is composed of multiple core modules, including Explore Destinations, Booking Management, Admin Dashboard, Travel Tools, and Notifications. Each module is designed to operate independently while maintaining seamless integration with other components through well-defined interfaces. These modules interact with the Firebase backend for data storage and retrieval, external APIs for services such as weather and maps, and the user interface for real-time user interaction.

These design models are essential for

- Ensuring smooth module integration
- Facilitating future expansion such as AI
- Keeping the architecture scalable
- Supporting consistent backend communication

Sequence Diagram

In the NorthPak application, the hotel booking process starts when the user opens the Explore page and selects a city like Hunza. The system shows a list of available hotels from Firebase. The user selects a hotel to see its details such as price, amenities, and availability. After that, the user enters booking information like dates and number of guests.

The app checks room availability. If rooms are available, the user clicks “Book Now.” The system then creates a booking record in Firebase with user details and booking information. A confirmation notification is sent to the user. Finally, the user can view the booking details and status in the app. [5]

User Basic Flow

This Figure 4.4 shows the complete flow of how a user interacts with the NorthPak app from opening the app, logging in, and navigating through modules. It illustrates how the user explores cities, hotels, tours, or attractions and optionally adds items to their Wishlist. The diagram also covers accessing travel tools and viewing profile sections such as bookings, reviews, FAQs, and account settings. Finally, it ends with the user logging out and being redirected to the welcome screen.

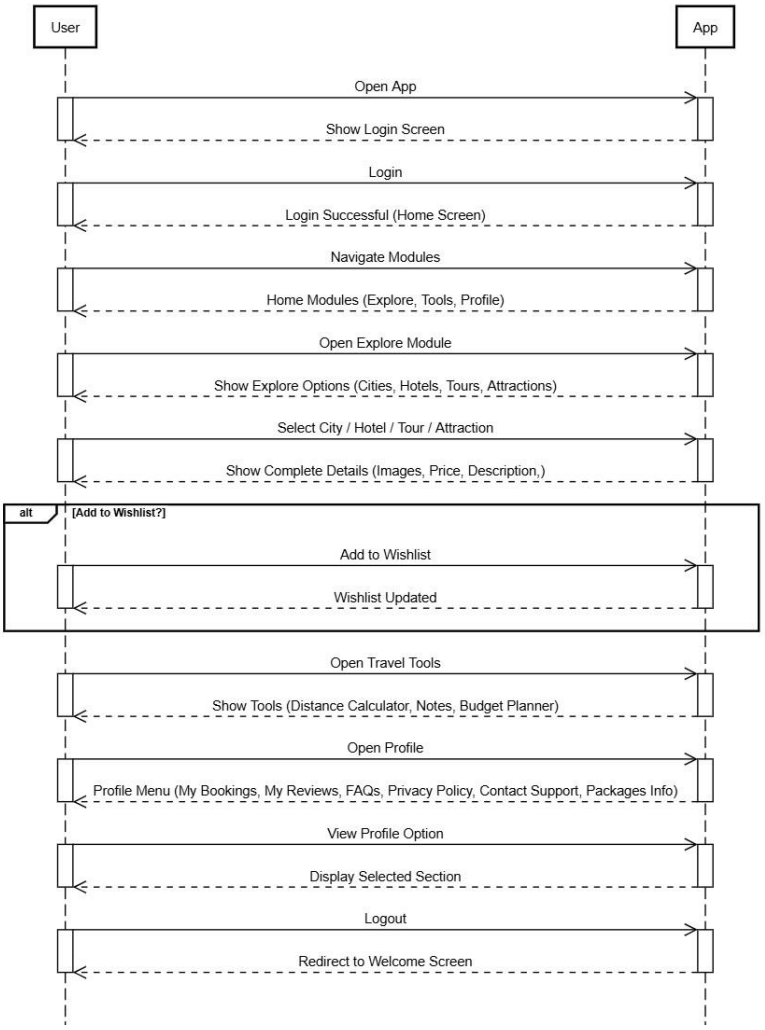

Figure 4.4 User Flow

Booking Sequence

This Figure 4.5 illustrates the complete booking workflow between the User, App, and Admin. The user selects a hotel, tour, or package, and for hotel bookings, the app first checks room availability before proceeding. After a booking is created in a pending state, it is sent to the admin, who either approves or rejects it based on availability and details. Finally, the user is notified of the final booking status, completing the process.

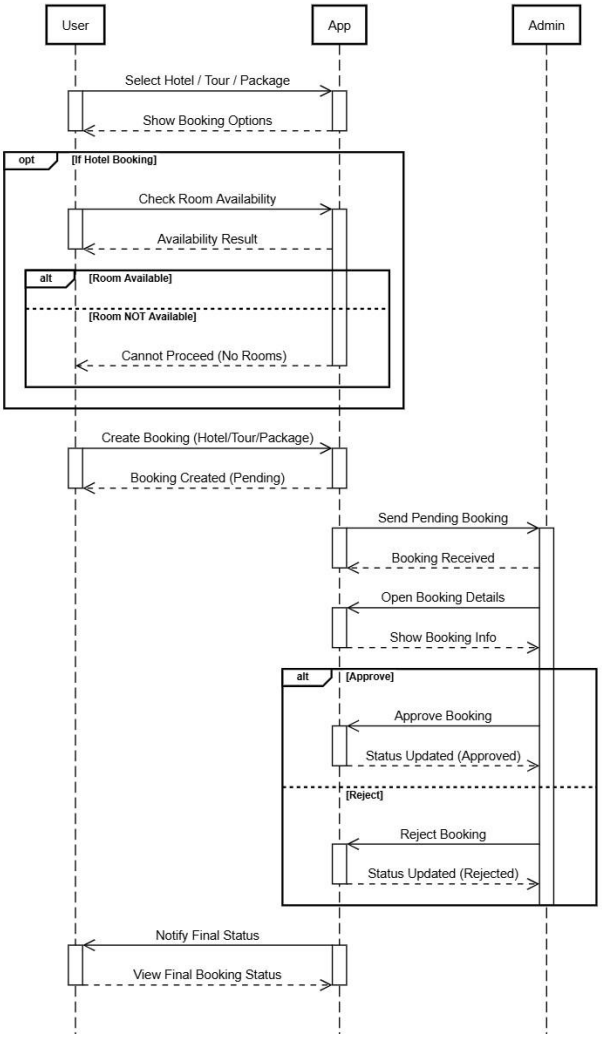

Figure 4.5 Booking Sequence

Review Flow

This Figure 4.6 shows how a user submits a review and how it flows through the system. After the user adds a review, the app forwards it to the admin for record keeping and moderation. The admin can then view all submitted reviews, and the updated review list is sent back to the app. Finally, the user can view the refreshed reviews displayed in the application.

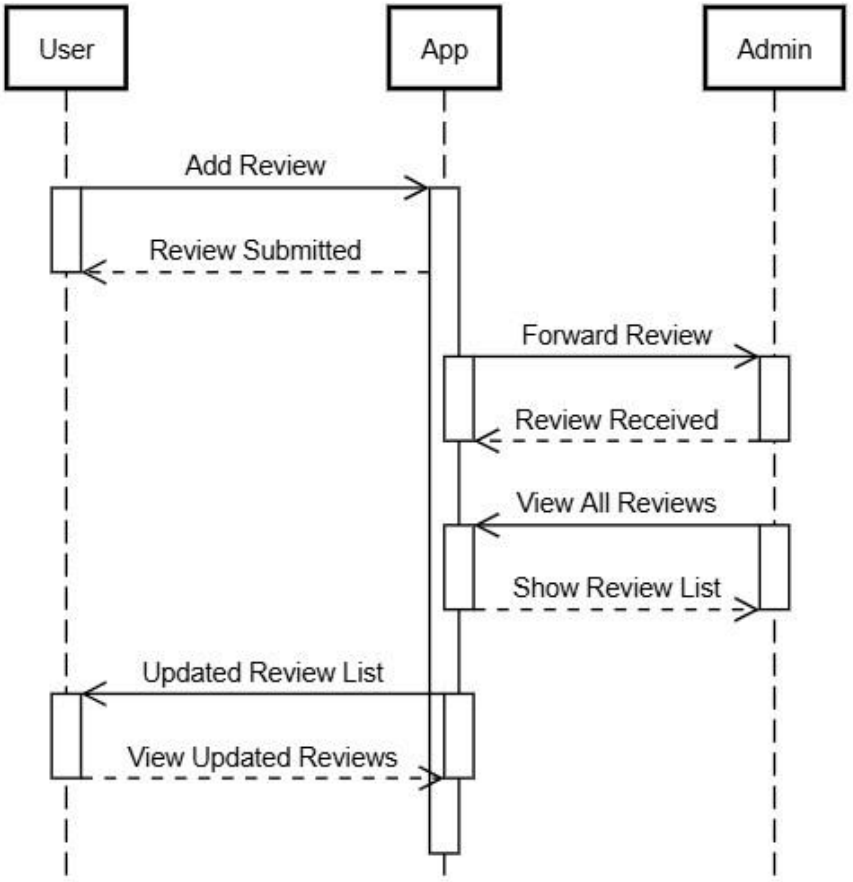

Figure 4.6 Review Flow

Admin Management Flow

This Figure 4.7 shows how the admin manages core system data through the app. The admin can add hotels, update packages and tours, modify room availability, and manage FAQs, with each update instantly reflected for the user. The app forwards all updates to the user interface so travelers always see the latest information. Finally, the admin logs out and is redirected to the welcome screen

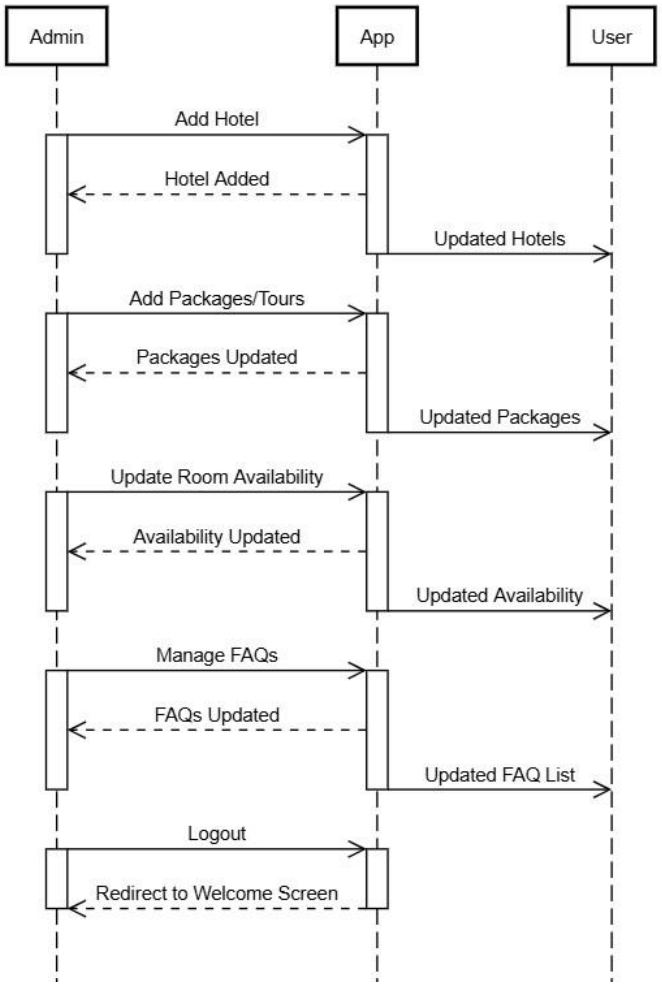

Figure 4.7 Admin Management

Class Diagram

The presented Figure 4.8 illustrates the overall class structure of the NorthPak Travel Application, providing a detailed representation of the system's core entities, their attributes, and the relationships between them. It defines how different components such as User, Booking, Hotel, Room, Tour, Package, and Admin are organized and interconnected to support the application's functionality. The diagram highlights both static data structures and the interactions between modules, enabling a clear understanding of how information flows within the system. Key features such as booking management, Wishlist handling, reviews, travel tools, and administrative control are modeled through well-defined classes and associations. This structured representation serves as a foundational blueprint for system development, ensuring consistency, scalability, and efficient communication between different parts of the application.

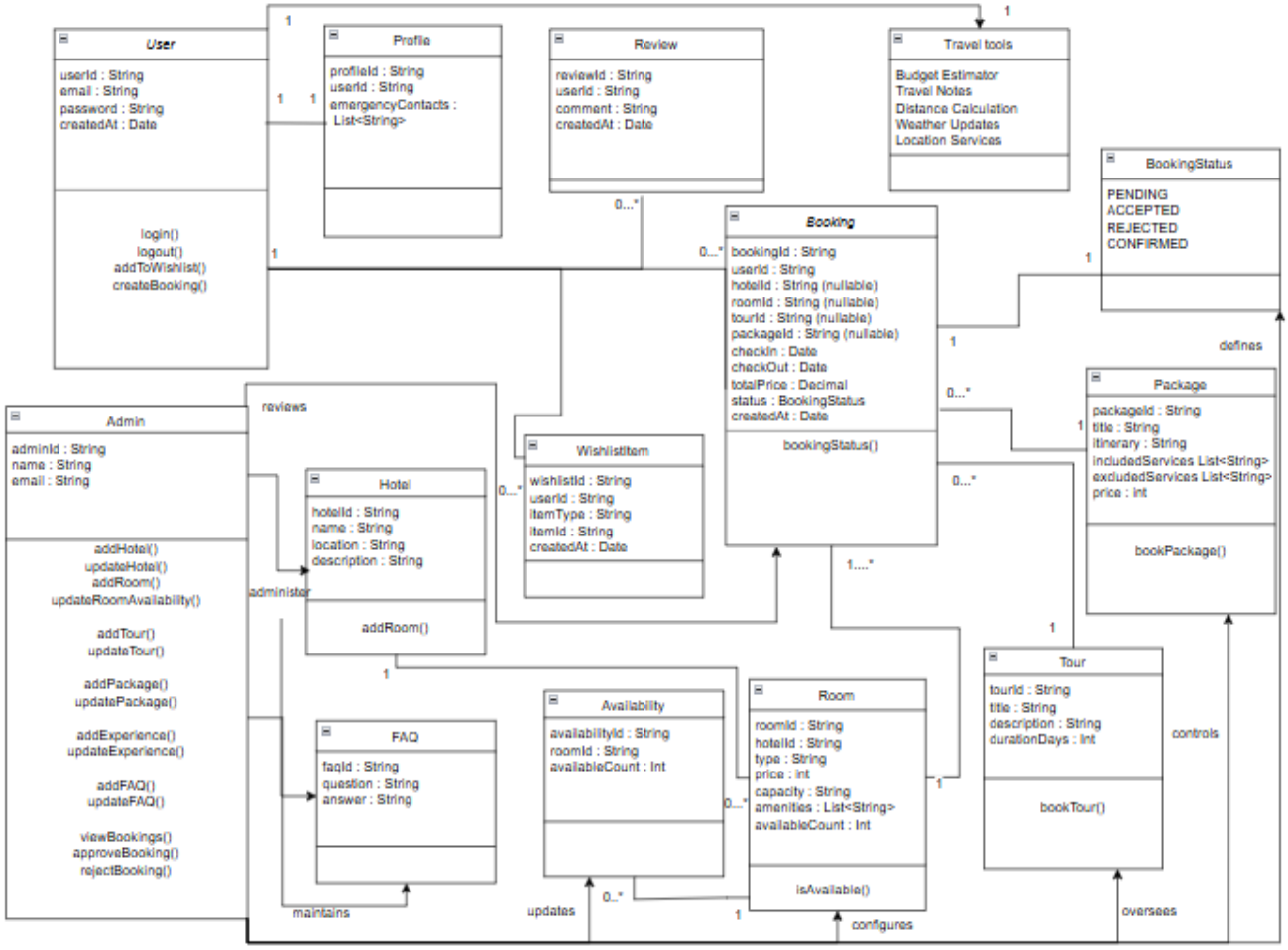

Figure 4.8 Class Diagram

Classes

Classes are the fundamental building blocks of an object-oriented system that represent real-world entities and their behavior. They encapsulate related data and functions to perform specific tasks within the application.

User

Represents a registered user of the application who can access travel services and perform bookings.
Attributes: userId, email, password
Functions: login(), logout(), createBooking(), addToWishlist()

Profile

Represents the personal information and preferences of a user stored within the system.
Attributes: profileId, userId, fullName,emergencyContacts, createdAt
Functions: updateProfile(), addEmergencyContact(), viewProfile()

TravelTools

Provides utility features to assist users in planning trips.
Includes budget estimation, travel notes, distance calculation, weather updates, and location services.

Booking + Booking Status

Represents reservation records in the system.
Attributes: bookingId, userId, hotelId, roomId, tourId, packageId, checkIn, checkOut, totalPrice, status, createdAt
Function: bookingStatus()
This is the central class that connects users with services such as hotels, tours, and packages.
Defines the state of a booking: Pending, Accepted, Rejected, Confirmed.

Hotel

Represents hotel information.
Attributes: hotelId, name, location, description
Function: addRoom()

Room

Represents a hotel room available for booking within the system. It stores room details such as type, price, capacity, and amenities.
Attributes: roomId, hotelId, type, price, capacity, amenities
Functions: viewDetails()

Wishlist Item

Represents an item saved by a user for future reference. It allows users to quickly access their preferred hotels, tours, or packages.
Attributes: wishlistId, userId, itemType, itemId, createdAt
Functions: addItem(), removeItem(), viewWishlist()

Tour

Represents travel tours.
Attributes: tourId, title, description, durationDays
Function: bookTour()

Package

Represents travel packages.
Attributes: packageId, title, itinerary, includedServices, excludedServices, price
Function: bookPackage()

FAQ

Stores frequently asked questions and represents accepted reviews.
Attributes: faqId, question, answer, reviewId, userId, comment, createdAt

Admin

Represents the system administrator.
Attributes: adminId, name, email
Functions: manage hotels, tours, packages, FAQs, bookings, and availability.

Relationships

Relationships define how different classes in the system interact and communicate with each other. They represent associations, dependencies, and connections between system components.

User Relationships

- A User has a one-to-one relationship with Profile.
- A User can have multiple Bookings (one-to-many).
- A User can write multiple Reviews and can have Wishlist Items (one-to-many).

Booking Relationships

- Each Booking has one Booking Status (many-to-one).
- A Booking may be associated with a Hotel, Room, Tour, or Package (optional relationships depending on booking type).

Service Relationships

- A Hotel contains multiple Rooms (one-to-many).
- A Hotel can have multiple Reviews (one-to-many).
- A Room is linked to Availability records (one-to-many).
- A Tour can be associated with multiple Bookings (one-to-many).
- A Package can be associated with multiple Bookings (one-to-many).

Administration Relationships

- Admin manages Hotels, Rooms, Tours, Packages, FAQs, and Bookings.
- Admin can approve or reject bookings and update room availability.

Chapter 5

Implementation

5. Implementation

The implementation phase involves developing the travel application by integrating all system modules into a functional platform. The app combines internal logic with external APIs, such as weather services and location-based features, to provide real-time information and recommendations. A user-friendly interface allows users to explore destinations, book hotels and tours, and use travel tools. Additionally, an admin panel is implemented to manage content, bookings, and overall system operations efficiently.

5.1 Algorithm

The following core technologies and algorithms are used in the development of NorthPak to implement its key travel and booking features:

5.1.1 Explore

Function: exploreCity(cityId)

Input: cityId

Output: cityContent

Pseudocode:

1. User selects a city card from the Explore screen.
2. Retrieve city-related data from the Firestore database:
    - Hotels
    - Tours
    - Food places
    - Sports activities
3. Fetch related travel content:
    - YouTube travel videos
    - External travel blogs
4. Organize the retrieved data into categories.
5. Display all city-related information on a single screen.
6. Return the city content.

5.1.2 User Authentication

Function: authenticateUser(username, password)

Input: username (string), password (string)

Output: user (object) or error (string)

Pseudocode:

1. Retrieve user record from the database based on the provided username.
2. If a user with the given username exists:
   a. Compare the provided password with the stored password hash.
   b. If the passwords match, return the user object.
   c. If the passwords do not match, return an error message indicating "Incorrect password".
3. If no user is found, return an error message indicating "User not found". [7]

5.1.3 Booking & Availability Management (User + Admin)

Function: handleBooking(userID, hotelID, dates)

Input: userID (integer), hotelID (integer), dates (date range)

Output: booking status (string)

Pseudocode:

1. User selects hotel and booking dates.
2. System checks room availability for selected dates.
3. If rooms are available:
   a. Create booking with status "Pending".
   b. Send request to admin for approval.
4. Admin reviews booking request:
   a. Check current room availability.
   b. If available:
   i. Approve booking.
   ii. Update room availability.
   iii. Change status to "Confirmed".
   c. Else:
   i. Reject booking.
   ii. Keep availability unchanged.
5. Notify user about booking status.

5.1.4 Admin Booking Management

Function: manageBooking(bookingID, decision)

Input: bookingID (integer), decision (Approve/Reject)

Output: success (Boolean)

Pseudocode:

1. Retrieve booking record using bookingID.
2. If decision is "Approve":
   a. Update booking status to "Confirmed".
   b. Enable payment option for the user.
3. Else if decision is "Reject":
   a. Update booking status to "Rejected".
4. Notify the user about the booking status.

5.1.5 Distance Calculation

Function: calculateDistance(lat1, lon1, lat2, lon2)

Input: coordinates (float)

Output: distance (float)

Pseudocode:

1. Convert latitude and longitude values from degrees to radians.
2. Apply Haversine formula to calculate distance.
3. Multiply result by Earth radius.
4. Return calculated distance.

5.1.6 Budget Estimator

Function: estimateBudget(days, hotelCost, travelCost, foodCost)

Input: days (integer), costs (float)

Output: total budget (float)

Pseudocode:

1. Calculate total hotel cost = days × hotelCost.
2. Calculate total food cost = days × foodCost.
3. Add travel cost.
4. Compute total budget = hotel + food + travel and return total budget

5.1.7 Weather Information Retrieval

Function: getWeather(location)

Input: location (string)

Output: weather data or error

Pseudocode:

1. Send request to weather API using location.
2. Receive response from API.
3. If response is valid:
   a. Extract temperature, weather condition, and humidity.
   b. Return weather data.
4. Else:
   a. Return error message "Weather data not available". [8]

5.1.8 Review and Rating System

Function: submitReview(userID, hotelID, rating, comment)

Input: userID, hotelID (integer), rating (integer), comment (string)

Output: success (Boolean)

Pseudocode:

1. Check if the user is authenticated.
2. Store review and rating in the database.
3. Retrieve current average rating and total reviews.
4. Update average rating using new rating.
5. Save updated rating.
6.

5.1.9 FAQ Management

Function: manageFAQ(action, faqData)

Input: action (Add/Update/Delete), faqData (object)

Output: success (Boolean)

Pseudocode:

1. If action = "Add", insert FAQ into database.
2. Else if action = "Delete", remove FAQ from database.

5.1.10 Hotel Management (Admin Panel)

Function: manageHotel(action, hotelData)

Input: action (Add/Update/Delete), hotelData (object)

Output: success (Boolean)

Pseudocode:

1. If action is "Add":
   a. Insert new hotel details into the database (name, location, price, rooms, amenities).
   b. Initialize room availability.
2. Else if action is "Update":
   a. Retrieve existing hotel record.
   b. Update hotel details (price, rooms, amenities, availability).
3. Else if action is "Delete":
   a. Remove hotel record from database.
4. Return success message "Hotel operation completed".

5.1.11 Content Management (Admin Panel)

Function: manageContent(action, contentType, contentData)

Input: action (Add/Update/Delete), contentType (Tour/City/Hotel/Package), contentData (object)

Output: success (Boolean)

Pseudocode:

1. Identify the content type (Tour, City, Hotel, or Package).
2. If action is "Add":
   a. Insert new content into the database based on contentType.
3. Else if action is "Update":
   a. Retrieve existing content record.
   b. Modify the required fields (price, details, availability, etc.).
   c. Save updated data to the database.
4. Else if action is "Delete":
   a. Remove the selected content from the database.
5. Sync updated data with frontend:
   a. Ensure changes are reflected in user application immediately.

5.2 External APIs

The following table presents the external APIs and services used in the application, along with their description, purpose of usage, and the functions or classes where they are implemented.

Table 5.1 External Api

Name of API / Service	**Description**	**Purpose**	**Function Used**
Open Weather API	Provides real-time weather data such as temperature, humidity, and conditions.	To display live weather updates for travel planning.	getWeather(location), WeatherService
Google Maps (External Integration)	A navigation service that provides maps and directions.	To allow users to open locations in Google Maps for navigation.	openMap(latitude, longitude), URLLauncher

5.3 User Interface

The user interface of the application is designed to be simple, interactive, and user-friendly to ensure a smooth experience for users. It allows users to easily navigate through different features such as exploring destinations, booking hotels and tours, checking weather updates, and using travel tools. The interface is developed using Flutter and provides a responsive layout that works efficiently across different devices. [9]

5.3.1 Splash Screen

This screen represents the initial loading interface of the application. It displays the app logo and branding along with a travel-themed illustration resources in the background.

Figure 5.1 Splash Screen

5.3.2 Welcome and Role Selection Screen

This screen allows users to choose between Admin and User modes. It provides two buttons: “Continue as Admin” and “Continue as User.”

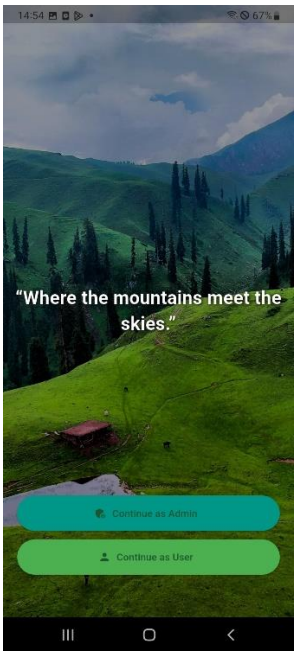

Figure 5.2 Welcome Screen

5.3.3 User Login Screen

The login screen enables users to authenticate themselves using email and password. It also includes options for sign up and forgot password, improving.

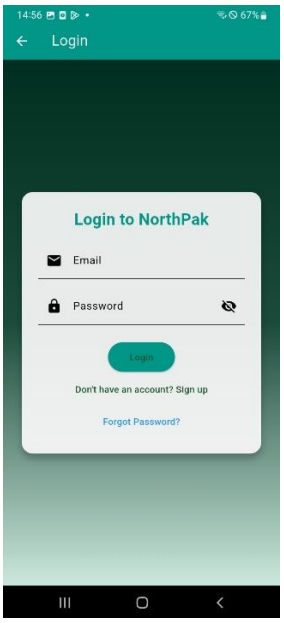

Figure 5.3 User Login

5.3.4 Admin Login Screen

Similar to the user login, this screen is specifically for administrators. It ensures restricted access to admin

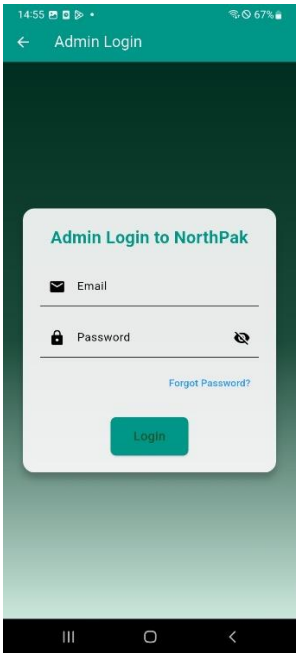

Figure 5.4 Admin Login

5.3.5 Explore Home Screen

This is the main dashboard for users. It displays different tourist destinations. A search bar is also provided for easy navigation.

Figure 5.5 Explore Home Screen

5.3.6 Destination Detail Screen

This screen shows categorized attractions such as Cultural Tours, Museums & Attractions. Each category contains visually rich cards with images and titles.

Figure 5.6 Destination Detail Screen

5.3.7 Attraction Detail & Booking Form

This screen displays detailed information about a selected attraction along with a booking form popup. This enables direct booking of experiences.

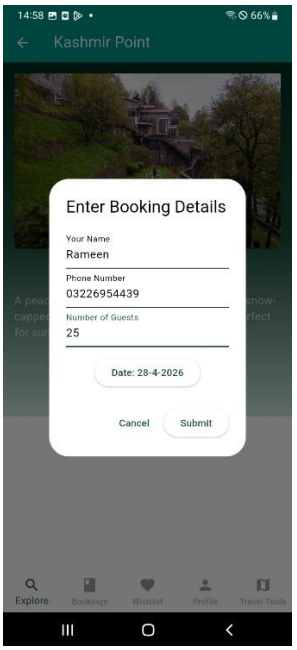

Figure 5. 7 Tour Booking

5.3.8 Hotel Detail & Room Listing

This screen shows available rooms in a selected hotel. Users can choose rooms based on their preference as all the facilities are listed clearly for each room type.

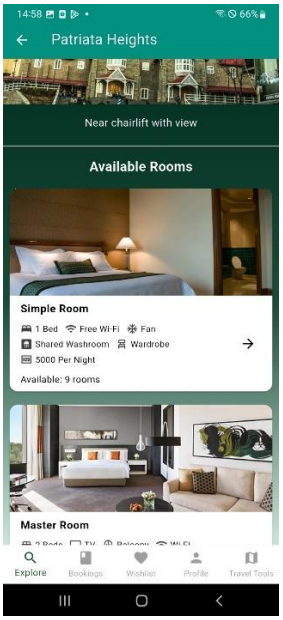

Figure 5.8 Hotel Details

5.3.9 Room Booking Screen

This screen collects booking details such as name and phone number. After submission, a confirmation dialog appears, ensuring the user that their booking request has been received.

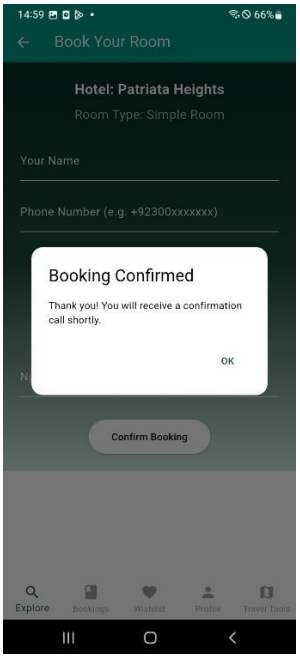

Figure 5 9 Room Booking

5.3.10 My Bookings Screen

This screen displays all user bookings categorized. Each booking shows details like date, guests, and status (Accepted / Pending). Users can also cancel bookings.

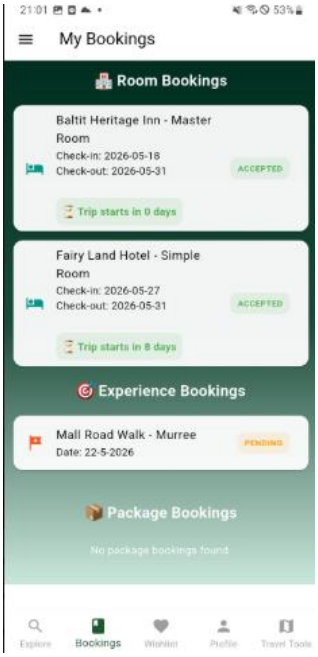

Figure 5.10 My Bookings

5.3.11 Weather Screen

This screen provides real-time weather information for selected locations (e.g., Swat). This helps users plan trips effectively.

Figure 5.11 Weather Screen

5.3.12 Location Services Screen

This screen integrates map functionality to display the user's current location. It also provides quick navigation options to northern cities.

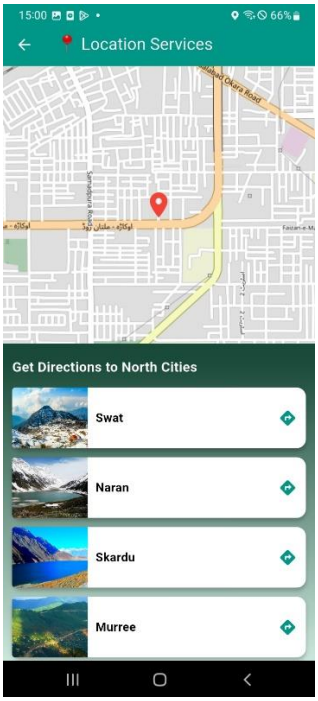

Figure 5. 12 Location Services

5.3.13 Budget Estimator Screen

This feature allows users to estimate travel costs. Inputs include Number of days, Travel mode, Room type and Daily expenses. The system calculates and stores budget estimates for future reference.

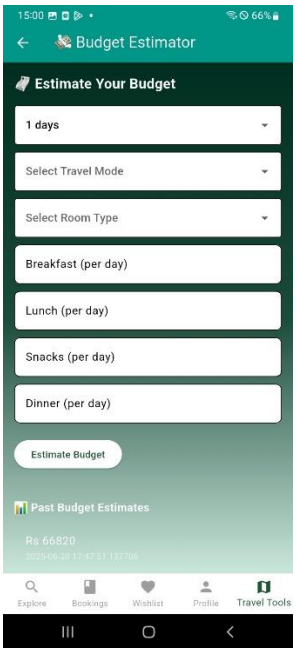

Figure 5.13 Budget Estimator

5.3.14 Travel Media Hub Screen

Users can watch travel videos and read blogs/guides related to famous tourist destinations.

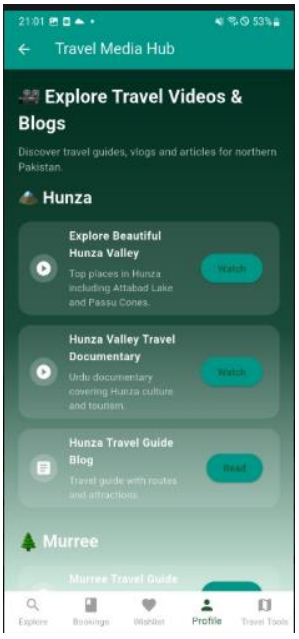

Figure 5.14 Travel Media Hub

5.3.15 Distance & Duration Screen

This feature calculates the distance and travel time between two cities. Users can also save routes for later use. It improves travel planning efficiency.

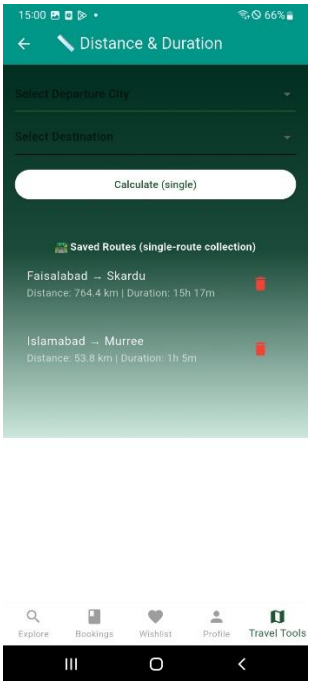

Figure 5.15 Distance and Duration Screen

5.3.16 Package Detail Screen

This screen provides details about travel packages including. It helps users understand complete tour offerings before booking.

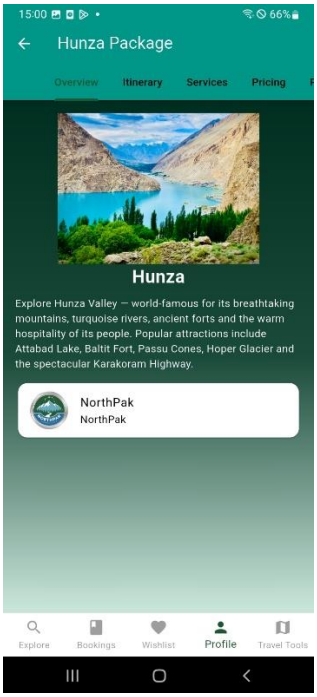

Figure 5. 16 Package Details

5.3.17 Profile Screen

The profile section includes Push notification toggle, Review submission, App information (FAQ, About, Privacy) and Emergency contacts. It centralizes user settings and support features.

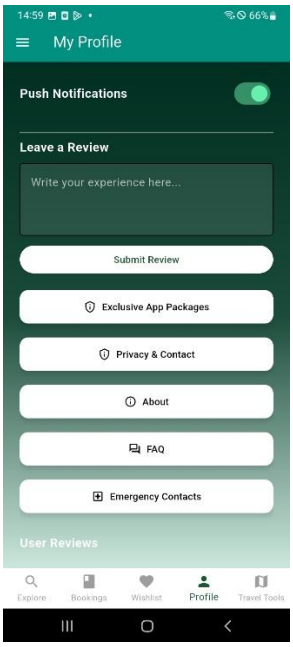

Figure 5. 17 Profile Screen

5.3.18 Travel Checklist Screen

Users can organize and track important travel essentials before starting their trip.

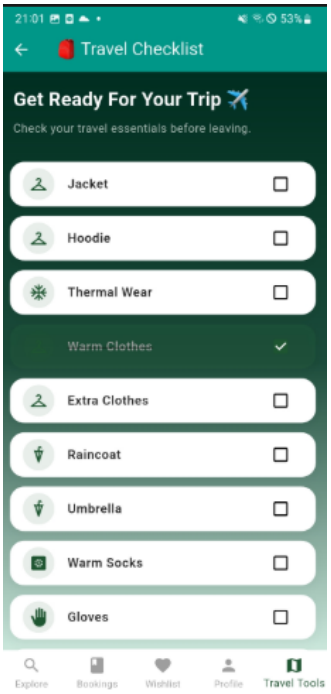

Figure 5.18 Travel Checklist

5.3.19 Admin Panel

This screen shows admin functionalities such as: Add Experience, Add Hotel, Add FAQ Manage Bookings, View Reviews and Manage Packages It provides full control over app content and operations.

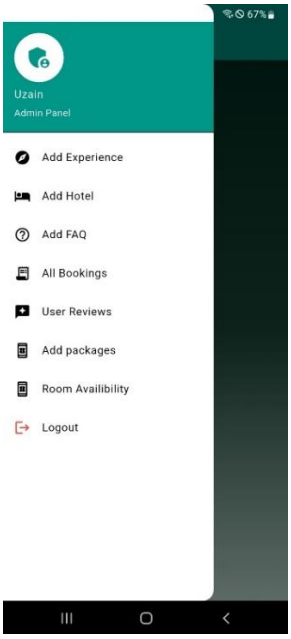

Figure 5.19 Admin Panel

5.3.20 Admin Reviews Screen

Enables admins to monitor user feedback, approve or reject pending reviews.

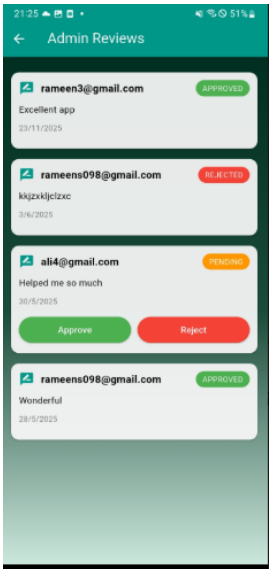

Figure 5.20 Admin Reviews Screen

5.3.21 Admin – All Bookings Screen

This screen displays all bookings from users. Admins can:

- View booking details
- Approve or reject bookings
- Manage booking status

This is crucial for backend management.

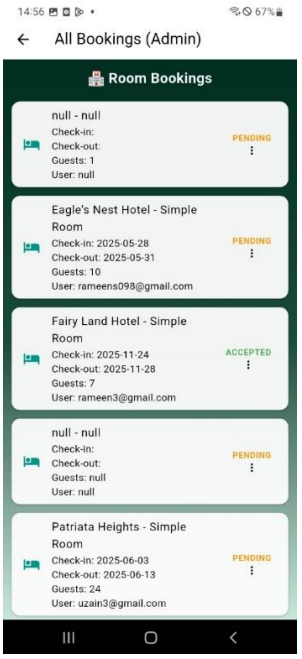

Figure 5.21 Admin Booking Management

5.3.22 Add Hotel Screen

Allows the admin to select a city, enter hotel details, upload image paths, and manage hotel listings with delete functionality.

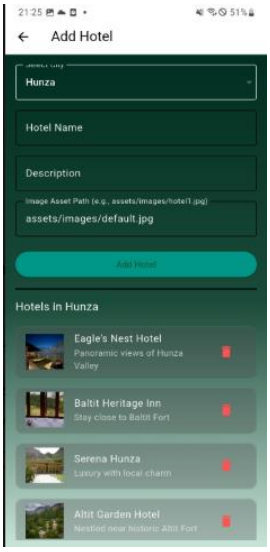

Figure 5.22 Add Hotel Screen

Chapter 6

Testing and Evaluation

6. Testing and Evaluation

Testing and evaluation were conducted to ensure that the NorthPak Travel Application functions according to the requirements defined in the. The goal of this phase was to verify that the implemented system meets the defined functional and non-functional requirements and that all modules interact correctly according to the system architecture.

The testing process focused on validating the core features including user authentication, destination exploration, hotel listings, tour booking, review submission, and administrative management functions. Both manual and automated testing techniques were used to ensure the reliability, usability, and correctness of the application.

6.1 Manual Testing

Manual testing was performed to evaluate whether the implemented features behave according to the requirements. During manual testing, testers interacted with the mobile application directly and executed different test scenarios based on the use cases and functional requirements

The testing process covered the major modules including:

- User authentication module
- Destination browsing module
- Hotel listing module
- Tour booking module
- Review and feedback module
- Admin management module

Each module was tested with different input scenarios to verify that the system behaves correctly and produces the expected results. Manual testing also helped identify usability issues, navigation errors, and incorrect system responses.

6.1.1 System Testing

Once the system has been successfully developed, testing is performed to ensure that it works as intended. This process verifies that the system meets all the requirements defined earlier. System testing also helps in identifying errors that may not be visible during development.

Different types of testing were conducted, including **unit testing, functional testing, and integration testing**. Unit testing ensures that individual components of the system function correctly. Functional testing verifies that all features, such as booking, filtering, and weather updates, operate as expected. Integration testing checks whether different modules of the system work together smoothly.

All testing activities were completed before the system was deployed to ensure reliability, accuracy, and a smooth user experience.

6.1.2 Unit Testing

Unit testing was performed to verify individual components or modules of the NorthPak Travel App in isolation. The purpose of this testing was to ensure that each function works correctly on its own before being integrated with other parts of the system.

In this project, unit testing focused on small functional units such as login validation, distance calculation, budget estimation, data filtering, and API response handling. Each function was tested separately by providing specific inputs and checking whether the expected outputs were produced. For example, the login validation function was tested with valid and invalid credentials to ensure correct authentication behavior, while the budget estimator was tested with different input values to verify accurate calculations.

By isolating each module, unit testing made it easier to detect and fix errors at an early stage of development. It helped identify issues such as incorrect logic, calculation errors, and improper handling of inputs. This reduced the chances of bugs appearing in later stages, especially during integration and system testing.

Additionally, unit testing improved code reliability and maintainability. Since each module was verified independently, developers could confidently update or modify code without affecting other parts of the system. [10]

User Authentication Module

The Table 6.1 below shows this module verifies that users can securely register, log in, and log out using valid credentials. Ensures proper validation, error handling, and protection of user data during authentication processes.

Table 6.1.1 User Authentication Module

No.	Test Case / Test Script	Attribute and value	Expected Result	Result
1	Validate user login	Username: user1 Password: 1234	Login successful with valid credentials	Pass
2	Invalid login attempt	Username: user1 Password: wrong	Error message displayed	Pass

Weather API Module

The Table 6.2 below shows this module fetches real-time weather data from an external API based on the user's selected location. Ensures accurate display of weather conditions and proper handling of API responses and errors

Table 6.2 Weather Api Module

No.	Test Case / Test Script	Attribute and value	Expected Result	Result
1	Fetch weather data	Location: Hunza	Weather data retrieved successfully	Pass
2	Invalid location input	Location: XYZ	Error or no data handled properly	Pass

Budget Calculator Module

The Table 6.3 below shows this module calculates the estimated travel cost based on user inputs such as destination, duration, and selected services. Ensures accurate budget estimation and helps users plan trips according to their financial constraints.

Table 6.3 Budget Calculator Module

No.	Test Case / Test Script	Attribute and value	Expected Result	Result
1	Calculate budget	Days:3 Location: Murree	Estimated budget calculated correctly	Pass
2	Empty input handling	Days: null	Validation message displayed	Pass

Review System Module

The Table 6.4 below shows this module allows users to submit ratings and reviews for destinations, hotels, and services. Ensures feedback is stored correctly and displayed to help other users make decisions.

Table 6.4 Review System Module

No.	Test Case / Test Script	Attribute and value	Expected Result	Result
1	Submit review	Comment: Good	Review saved successfully	Pass
2	Empty review submission	Comment: null	Validation error displayed	Pass

Room Availability Module

The Table 6.5 below shows this module checks and displays available rooms based on user selection and booking dates. Ensures real-time updates of room status after booking or cancellation.

Table 6.5 Room Availability

No.	Test Case / Test Script	Attribute and value	Expected Result	Result
1	Check available rooms	Hotel ID: H01	Available rooms displayed	Pass
2	No rooms available	Hotel full	"No rooms available" message shown	Pass

6.1.3 Functional Testing

Functional testing was performed to evaluate the complete functionality of the system by testing user interactions and overall system behavior. The primary goal of this testing was to ensure that all features of the application operate according to the specified requirements from the user's perspective. In the NorthPak Travel App, functional testing focused on validating core features.

Booking Features

The Table 6.1 shows this module allows users to book hotels, tours, and travel services through the application by selecting destinations, dates, and preferences.

It processes booking details, updates availability, and stores booking information while providing confirmation and status updates to the user.

Table 6.6 Booking features

No.	Test Case / Test Script	Attribute and value	Expected Result	Result
1	Book day tour	Tour:Hunza Date selected	Booking request submitted successfully	Pass
2	Book travel package	Package: Skardu Trip	Package booked successfully	Pass
3	Book hotel room	Room selected	Room booked and availability updated	Pass

User Features

The Table 6.7 below shows this module provides users with functionalities such as registration, login, profile management, and browsing destinations. Includes functionalities such as viewing weather, calculating budget, submitting reviews, and writing notes. Ensures that user inputs are processed correctly successfully.

Table 6.7 User Features

No.	Test Case / Test Script	Attribute and value	Expected Result	Result
1	View weather	Location selected	Weather displayed correctly	Pass
2	Calculate budget	Input values entered	Budget displayed	Pass
3	Submit review	Rating + Comment	Review displayed	Pass
4	Write travel notes	Note entered	Note saved successfully	Pass
5	Create travel checklist	Checklist items entered	Checklist saved and displayed correctly	Pass

Navigation & Support Features

The Table 6.8 below shows this module enables users to easily navigate through the app and access different sections smoothly. Provides support options such as help, FAQs, and contact features to assist users when needed.

Table 6.8 Navigation and Support Features

No.	Test Case / Test Script	Attribute and value	Expected Result	Result
1	Open Google Maps	Click location	External map opens	Pass
2	View emergency contacts	Open section	Contacts displayed	Pass
3	View FAQs	Open FAQ page	FAQs displayed properly	Pass

6.1.4 Integration Testing

Integration testing was conducted to verify the interaction between different modules of the application. The main objective of this testing was to ensure that all integrated components—such as the user interface, backend services, database, and external APIs work together correctly and exchange data without errors.

During integration testing, different modules were combined and tested as a group rather than individually. For example, the connection between the mobile application interface and backend services was tested to ensure that user requests were properly processed and responses were accurately displayed. Similarly, the integration between the authentication system and database was verified to confirm that users could successfully log in, and their data was correctly retrieved and stored.

Special attention was given to data flow between components. This included testing whether data entered by users was correctly transmitted to the database and whether the retrieved data was displayed accurately in the application. API integrations, such as weather data or location-based services, were also tested to ensure proper communication, correct responses, and error handling in case of network failures. Integration testing also helped identify issues such as mismatched data formats, broken links between modules, delayed responses, and failures in communication between frontend and backend systems. By resolving these issues, the system achieved better reliability and a smoother user experience.

User & Booking Integration

The Table 6.9 below shows this module ensures that user actions such as selecting and booking services are properly linked with their account. Verifies that booking data is correctly stored, retrieved, and associated with the respective user profile.

Table 6.9 User and Booking Integration

No.	Test Case / Test Script	Attribute and value	Expected Result	Result
1	Login with database	Username, Password	User authenticated and dashboard loaded	Pass
2	Tour and Package booking	Tour, Date	Booking stored	Pass

External Services Integration

The Table 6.10 below shows this module integrates third-party services such as weather APIs, maps, and location services into the application. [11]

Table 6.10 External services integration

No.	Test Case / Test Script	Attribute and value	Expected Result	Result
1	Weather API integration	Location	Live weather data displayed	Pass
2	Google Maps integration	Location clicks	Map opens externally	Pass

Admin & System Integration

The Table 6.11 below shows this module ensures proper coordination between admin panel and system modules.

Table 6.11 Admin and system Integration

No.	Test Case / Test Script	Attribute and value	Expected Result	Result
1	Admin manage tours	Add/Edit/Delete	Changes reflected to users	Pass
2	Admin manage hotels	Update rooms	Availability updated in system	Pass
3	Admin manage reviews	Approve/Delete	Reviews updated on UI	Pass

6.2 Automated Testing

Automated testing was performed to verify system operations efficiently using development tools and built-in testing frameworks. These tests were designed to execute predefined test cases automatically without manual intervention, ensuring consistency and accuracy in repeated executions. Automated testing is particularly useful for validating core functionalities that are frequently used within the application.

In this project, automated tests were applied to critical modules such as user authentication, data retrieval from the database, API responses, and backend service interactions. For example, login authentication was tested multiple times with different input combinations to ensure correct validation, error handling, and security. Similarly, database operations such as storing, updating, and retrieving user data were automatically tested to verify data integrity and consistency.

One major advantage of automated testing is its ability to run the same tests repeatedly with high speed and precision. This helped ensure that system functions remained stable even after multiple executions or updates to the application. It also reduced human error and saved time compared to manual testing.

Additionally, automated testing helped in identifying performance-related issues, such as slow response times in API calls or delays in data retrieval from the database. It also ensured that backend services, such as Firebase integration, functioned reliably under different conditions.

Overall, automated testing improved the reliability, efficiency, and scalability of the system by continuously validating its core operations and ensuring that any changes or updates did not break existing functionality.

Tools Used

The development of the NorthPak Travel App involved the use of various tools and technologies to ensure efficient design, implementation, testing, and documentation of the system. These tools supported different stages of the software development lifecycle, from coding and backend integration to testing and report preparation. Each tool was selected to improve productivity, accuracy, and overall system performance.

Development Tools

- **Flutter (SDK):** Used to develop the cross-platform mobile application with a single codebase. It provided a rich set of UI components and ensured smooth performance.
- **Dart Programming Language**: Used as the core language for implementing application logic and functionality.

Backend Services

- **Firebase Authentication:** Used for secure user login and registration functionality.
- **Firebase Firestore:** A cloud-based NoSQL database used to store and manage application data such as user information, bookings, and reviews.

Testing Tools

- **Flutter Testing Framework:** Used for performing unit and widget testing to verify individual components.
- **Manual Testing:** Conducted to validate user interactions and overall system behavior.

Documentation Tools

- **Microsoft Word:** Used for writing and formatting the project report.
- **Microsoft PowerPoint:** Used to create the project presentation slides.

Other Tools

- **Android Studio:** Used as the development environment (IDE) for writing and debugging code.
- **Google Maps API and Weather API:** Used to provide location-based services and real-time weather information.

Chapter 7

Conclusion and Future Work

7. Conclusion and Future Work

This chapter summarizes the overall outcomes of the NorthPak Travel Application project and discusses potential improvements that can be implemented in the future.

7.1 Conclusion

The NorthPak Travel Application was developed to address the challenges faced by tourists when planning trips to the northern areas of Pakistan, travelers often struggle to find reliable and centralized information about tourist destinations, accommodations, and travel services. Existing platforms provide limited localized information, which makes travel planning more difficult and time consuming.

To solve this problem, the project introduced a mobile-based tourism management system that integrates multiple travel related services into a single platform. The application allows users to explore tourist destinations, view hotel listings, book tours, calculate travel budgets, access weather and route information, and share reviews with other travelers.

During the development process, the system architecture and design models defined implemented using modern development technologies. The mobile application interface was developed using Flutter, while backend services such as authentication, database storage, and real time data synchronization were managed using Firebase. External APIs were also integrated to provide additional travel related services such as location navigation and weather information.

The system was thoroughly tested using various testing techniques including unit testing, functional testing, system testing, and integration testing

The results of the project demonstrate that the NorthPak Travel Application successfully fulfills its primary objectives.In addition, the system offers administrative features that allow administrators to manage hotels, tours, and frequently asked questions, ensuring that the information available to users remains accurate and up to date.

Overall, the project contributes to the digital transformation of tourism services in Pakistan by providing a modern mobile application that improves travel planning and enhances the tourism experience for both local and international travelers

7.2 Future Work

Although the NorthPak Travel Application successfully implements the core features, there are several opportunities for future enhancements that can further improve the system's functionality, scalability, and user experience.

One potential improvement is the integration of online payment systems. Currently, the application allows users to view and book tours, but future versions of the system could include secure payment gateways that enable users to complete transactions directly within the application.

Another possible improvement is the implementation of advanced recommendation systems. By incorporating machine learning algorithms, the application could analyze user preferences and previous searches to provide personalized travel recommendations. This would help users discover destinations, hotels, and tours that match their interests.

The system can also be improved by adding offline functionality. Many northern areas of Pakistan have limited internet connectivity, which can affect application usability. Future versions of the application could allow users to download maps, destination details, and travel information for offline access.

Additionally, the system could be expanded to include transportation services integration, such as bus schedules, car rental options, and ride-sharing services. This would provide travelers with a more complete travel planning platform.

Future development could also focus on improving the administrative management system by introducing advanced analytics tools that help administrators monitor tourism trends, analyze user behavior, and manage tourism services more efficiently.

Finally, the application could be extended to support multiple languages to make the platform more accessible for international tourists visiting Pakistan.

With these improvements, the NorthPak Travel Application has the potential to evolve into a comprehensive digital tourism platform that supports travelers, promotes local tourism businesses, and contributes to the sustainable development of tourism in Pakistan

References

[1] "Pakistan Tourism Development Corporation," [Online]. Available:https://www.ptdc.pk.

[2] "Northern Areas Travel Reviews and Guides.," [Online]. Available: https://www.tripadvisor.com/.

[3] "Software Development Life Cycle (SDLC)," [Online]. Available: https://www.tutorialspoint.com/sdlc/index.htm.

[4] "draw.io – Online Diagramming Tool.," [Online]. Available: https://app.diagrams.net.

[5] "Google, Cloud Firestore Documentation," [Online]. Available: https://firebase.google.com/docs/firestore .

[6] "SequenceDiagram.org, Online Sequence Diagram Tool," [Online]. Available: https://sequencediagram.org .

[7] "Google, Firebase Authentication," [Online]. Available: https://firebase.google.com/docs/auth .

[8] "OpenWeather, OpenWeather API," [Online]. Available: https://openweathermap.org/api

[9] "Google, Flutter Documentation," [Online]. Available: https://docs.flutter.dev.

[10] "Google, Android Studio User Guide," [Online]. Available: https://developer.android.com/studio .

[11] "Google, Google Maps Platform," [Online]. Available: https://developers.google.com/maps.